



Graph-Augmented Retrieval-Augmented Generation for Vietnamese Labor Law Question Answering

Thesis Report

Supervisor: Dr. Tran Hoang Tung

Student:

Tran Hien Chuong 22BA13056

Academic Year 2025–2026

Abstract

TODO: Write a concise 200–300 word abstract after chapter drafting. Mention the scoped Vietnamese labor-law corpus, hierarchy-aware chunking, Qdrant, Neo4j, graph-augmented retrieval, extractive citation-grounded answering, deterministic rule-based validation, and the constructed 69-query benchmark.

Acknowledgements

TODO: Add acknowledgements before submission.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Research Objectives	1
1.4 Scope of the Project	1
1.5 Contributions	1
1.6 Thesis Structure	2
2 Background and Related Work	3
2.1 Large Language Models	3
2.2 Retrieval-Augmented Generation	3
2.3 Legal Question Answering and Domain Challenges	3
2.4 Knowledge Graphs and Graph-Augmented RAG	4
2.5 Evaluation Methods for RAG Systems	4
3 System Design and Methodology	5
3.1 Overall System Architecture	5
3.2 Legal Corpus and Data Sources	7
3.3 Legal Document Preprocessing and Enrichment	10
3.4 Legal Cross-Reference Extraction and Graph Design	12
3.5 Vector-based and Hybrid Retrieval	15
3.6 Query Routing and Graph Expansion Policy	17
3.7 Grounded Answer Generation with Legal Citations	19
4 Implementation	24
4.1 Technology Stack	24
4.2 Data Processing Implementation	25
4.3 Vector Index and Hybrid Retrieval Implementation	25
4.4 Neo4j Graph Implementation	26
4.5 Graph-Augmented Retriever Implementation	26
4.6 Grounded Answer Generator and Validation	26
4.7 Evaluation Scripts and Reproducibility	27
4.8 Example Use Cases	27

5	Evaluation and Results	29
5.1	Evaluation Setup and Benchmark Design	29
5.2	Retrieval Evaluation Methodology	31
5.3	Answer and Citation Evaluation Methodology	33
5.4	Retrieval Results	35
5.5	End-to-End Results	37
5.6	Case Studies	38
5.7	Error Analysis and Limitations	39
6	Discussion	42
6.1	Key Findings	42
6.2	Impact of Hierarchical Chunking	42
6.3	Benefits of Graph-Augmented Retrieval	42
6.4	Role of Normative Hierarchy in Legal QA	43
6.5	Citation Grounding and Hallucination Control	43
6.6	Practical Applications	43
6.7	Limitations	44
7	Conclusion and Future Work	45
7.1	Conclusion	45
7.2	Future Work	45
	References	46
A	List of Legal Documents	47
B	Chunk Schema	48
C	Graph Schema	49
C.1	Ontology Labels and Relationships	49
C.2	Constraints and Indexes	49
C.3	Diagnostic Cypher Queries	50
D	Benchmark Samples	51
D.1	Case LBR_001: Definition of Employee	51
D.2	Case LBR_010: Oral Contract Exception	51
E	Evaluation Metrics	52
E.1	Information Retrieval Metrics	52
E.2	End-to-End System Pass Logic	52
F	Reproducibility Commands	53
F.1	Rerun Retrieval-Mode Evaluation	53
F.2	Rerun End-to-End Evaluation	53
F.3	Optional: Regenerate Qdrant Vector Index	53
F.4	Optional: Rebuild Neo4j Legal Graph	54

G	Example Retrieved Contexts and Answers	55
G.1	Example Input Query	55
G.2	Retrieved Evidence Chunk	55
G.3	Validated Generated Output	55

List of Figures

2.1	Standard retrieval-augmented generation pipeline.	3
3.1	Overall architecture of the proposed graph-augmented RAG system. . . .	6
3.2	Legal data processing pipeline with hierarchy-aware chunking and meta- data preservation.	10
3.3	Conceptual schema of the Neo4j legal knowledge graph.	13
3.4	Graph-augmented retrieval flow combining hybrid seed retrieval and legal graph expansion.	20
3.5	Answer generation and deterministic citation validation flow.	23

List of Tables

3.1	Summary of the scoped Vietnamese labor law corpus and document meta-data.	8
3.2	Legal chunking and data integrity validation results.	11
3.3	Summary of loaded node and edge counts in the Neo4j legal knowledge graph.	14
4.1	Technology stack.	24
5.1	Corpus and index statistics.	30
5.2	Retrieval modes comparison table.	35
5.3	Category-level result table.	36
5.4	End-to-end evaluation summary table.	37
6.1	Limitations summary.	44
A.1	Scoped Vietnamese labor-law corpus documents.	47

Chapter 1

Introduction

1.1 Background

TODO: Introduce Vietnamese labor law as a structured, citation-heavy information access domain. Explain why users need answers that preserve article, clause, point, document, and normative-rank context rather than only keyword matches.

1.2 Problem Statement

TODO: State the concrete problem: retrieval-only legal QA can miss cross-referenced provisions, implementing guidance, appendices, and procedure rules, which can lead to incomplete or weakly grounded answers.

1.3 Research Objectives

TODO: List measurable objectives: build a Vietnamese labor-law Graph-Augmented RAG pipeline, construct a hierarchy-aware corpus and graph, retrieve citation-grounded evidence, generate extractive answers with legal citations, and evaluate on the constructed 69-query benchmark.

1.4 Scope of the Project

TODO: Define the project scope around the six indexed Vietnamese labor-law and labor-procedure documents. Clarify that the system is for information assistance within the indexed corpus and is not a substitute for professional legal advice.

1.5 Contributions

TODO: Summarize contributions: cleaned legal corpus, hierarchy-aware legal chunking, metadata enrichment, cross-reference extraction, Qdrant vector index, Neo4j legal knowledge graph, graph-augmented retrieval, citation-grounded extractive answer generation, and deterministic validation artifacts.

1.6 Thesis Structure

TODO: Briefly describe Chapters 2–7 after drafting: background and related work, methodology, implementation, evaluation, discussion, and conclusion with future work.

Chapter 2

Background and Related Work

2.1 Large Language Models

TODO: Explain transformer-based language models, their use in question answering, and their limitations for legal domains, including hallucination, context limits, and the need for source grounding. Add verified citations.

2.2 Retrieval-Augmented Generation

TODO: Move and expand the old RAG background here. Explain indexing, embeddings, retrieval, context assembly, and grounded generation. Cite foundational RAG work such as [1] and add any newer verified RAG references.

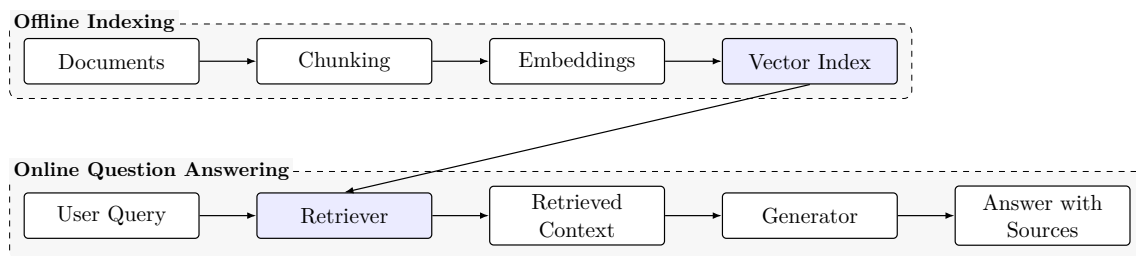


Figure 2.1: Standard retrieval-augmented generation pipeline.

2.3 Legal Question Answering and Domain Challenges

TODO: Legal QA requirements. Review legal QA requirements: jurisdiction specificity, temporal validity, citation traceability, cautious wording, and the risk of unsupported legal conclusions. Add legal QA references using [4].

TODO: Legal document structure and citation. Explain how Vietnamese legal documents are organized into documents, chapters, sections, articles, clauses, points, and appendices. Discuss why legal citations must preserve this hierarchy. Cite verified Vietnamese legal document sources using [2].

2.4 Knowledge Graphs and Graph-Augmented RAG

TODO: Knowledge graphs for legal retrieval. Introduce knowledge graphs for representing legal hierarchy, references, topics, actors, issue types, and normative relationships. Explain how graph structure complements embedding similarity. Add verified KG and legal KG references.

TODO: Graph-augmented retrieval. Move the old graph-augmented retrieval background here. Compare graph expansion with vector-only and hybrid retrieval, and cite GraphRAG or graph-augmented retrieval work using [3].

2.5 Evaluation Methods for RAG Systems

TODO: Review retrieval metrics, answer validation, and citation grounding. Distinguish human expert review, LLM-as-Judge, and deterministic rule-based validation; later chapters use deterministic rule-based validation only.

Chapter 3

System Design and Methodology

3.1 Overall System Architecture

Figure 3.1 illustrates the overall architecture of our graph-augmented Retrieval-Augmented Generation (GraphRAG) system, which is designed to support citation-grounded answers for Vietnamese labor-law questions. The main goal of this system is to retrieve accurate legal evidence from the indexed corpus and generate natural, reliable answers backed by precise citations.

A standard, flat RAG pipeline often fails in the legal domain because it misses crucial context. Vietnamese legal documents have a highly nested structure, typically organized as Document $\rightarrow$ Article $\rightarrow$ Clause $\rightarrow$ Point. A simple flat retrieval strategy splits these texts arbitrarily, which separates detailed clauses from their parent articles or documents. Furthermore, answering a single legal question often requires retrieving related decrees, circulars, appendices, or cross-referenced provisions that reside in different parts of the corpus. To preserve this legal structure and connect vector retrieval with graph expansion, our architecture combines semantic vector search with a legal knowledge graph.

The system is divided into two primary parts: an offline data processing pipeline and an online query-answering pipeline.

The offline ingestion pipeline processes raw legal documents into hierarchy-aware chunks. This process involves the following key stages:

1. **Text cleaning:** Normalizes unicode characters and removes OCR noise, headers, footers, and page numbers from the raw source files.
2. **Syntax-aware legal parsing:** Identifies legal structure boundaries (e.g., chapters, articles, clauses, and points) using regular expression patterns.
3. **Hierarchy-aware chunking:** Splits the document at leaf-level legal units (such as clauses or points) rather than using arbitrary word counts, preventing semantic fragmentation.
4. **Semantic Preservation Layer:** Formats the chunk contents and forces each leaf-level chunk to inherit metadata (such as document title, article number, and normative rank) from its parent elements.
5. **Metadata enrichment:** Attaches domain taxonomy metadata (such as legal topics, actors, and issue types) to each chunk.
6. **Reference extraction:** Automatically extracts relative and absolute cross-references

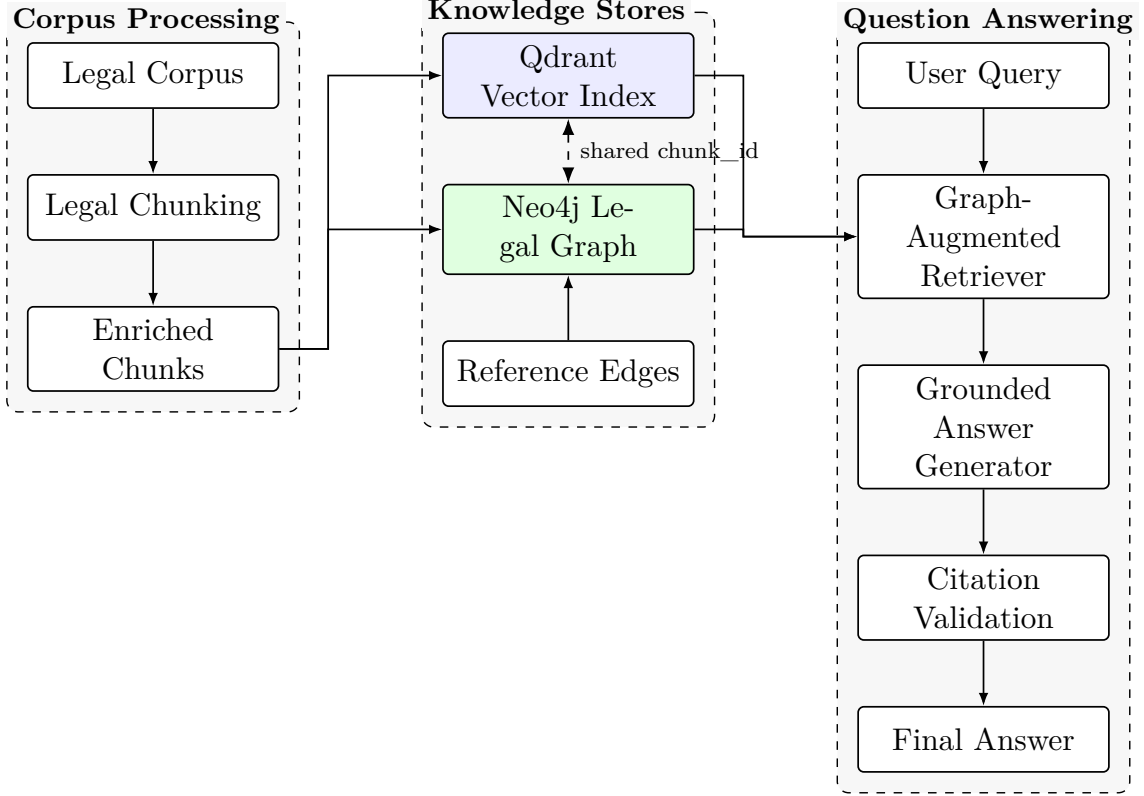


Figure 3.1: Overall architecture of the proposed graph-augmented RAG system.

mentioned within the text of each chunk.

Once the chunks are processed, they are indexed in two complementary knowledge stores: Qdrant and Neo4j. We use Qdrant to store and query dense semantic and sparse lexical embeddings for hybrid vector retrieval. Simultaneously, we load the document structures, metadata, and cross-references into Neo4j to build a legal knowledge graph. A shared `chunk_id` serves as the primary key that links the vector records in Qdrant with the corresponding evidence nodes in Neo4j, allowing us to connect vector retrieval with graph expansion at runtime.

At runtime, the online question-answering pipeline processes user queries and generates answers using the following sequence:

1. **User query q :** The user inputs a natural language question.
2. **Query routing:** The query is classified to identify target topics, actors, and specific article references.
3. **Hybrid seed retrieval:** The system performs dense semantic and sparse keyword search in Qdrant to find a set of initial seed chunks.
4. **Graph and reference expansion:** The system queries Neo4j using the retrieved seed chunks as entry points, traversing the graph to gather parent articles, related circulars, and implementing decrees.
5. **Reranking and context assembly:** The retrieved seed chunks and graph-expanded chunks are combined, deduplicated, and ranked using a semantic cross-encoder to select the most relevant contexts.

6. **Grounded answer generation:** The assembled contexts are passed to the Large Language Model (LLM) to synthesize a coherent response.
7. **Citation validation:** The draft answer and its inline citations are analyzed by a validation module to check whether citations are supported by retrieved contexts.
8. **Final response:** The system outputs the citation-grounded answer, or returns an explicit insufficient-context message if the retrieved sources do not support the query.

To formalize the context assembly step, the final context array C_{final} loaded into the generation layer is defined as:

$$C_{\text{final}} = \text{Rerank}(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}),$$

where S_k represents the set of seed chunks retrieved via hybrid search, C_{graph} denotes the additional context gathered through the Neo4j graph expansion, and C_{fallback} represents the coordinate and reference fallback contexts retrieved when specific article numerals are explicitly mentioned in the query.

After the generation step, the draft response passes through an answer validation layer. Instead of assuming the LLM’s output is always legally correct, this layer uses the retrieved **citation_text** and applies deterministic validation checks to reduce unsupported answers. Specifically, the validator checks that:

- All citations in the generated answer are present in the retrieved chunks.
- The generated article numbers are actually supported by the source text.
- Higher-rank legal contexts (such as Code provisions) are not ignored or contradicted by lower-rank circulars.
- An insufficient-context disclosure is activated if the retrieved documents do not contain enough evidence to answer the query.

This validation process helps prevent hallucinations and keeps the generated answers strictly grounded within the indexed corpus, without claiming universal legal correctness.

The subsequent sections of this chapter describe each of these components in detail. Section 3.2 outlines the characteristics of our legal corpus and metadata. Section 3.3 covers the document preprocessing and hierarchical chunking techniques. Section 3.4 explains the design and schema of the Neo4j legal knowledge graph. Section 3.5 formalizes our vector-based and hybrid retrieval strategies. Section 3.6 details the runtime online query routing and graph expansion policies. Finally, Section 3.7 describes the context reranking, budgeting rules, and the deterministic validation guardrails.

3.2 Legal Corpus and Data Sources

The performance, reliability, and precision of a Retrieval-Augmented Generation (RAG) system are fundamentally governed by the scope, structural integrity, and authority of its underlying knowledge base. In the domain of legal question answering, these parameters are especially critical: legal language is highly structured, dense, and semantically sensitive, where minor phrasing variations carry distinct statutory implications. Consequently, retrieval failures or LLM-based hallucinations in legal contexts can lead to incorrect advice, regulatory non-compliance, or severe procedural errors. To mitigate

these risks and establish a highly reliable, verifiable QA pipeline, this study utilizes an intentionally scoped legal corpus consisting of six key Vietnamese labor-related documents, summarized in Table 3.1.

Rather than attempting to index the entire, dynamic repository of Vietnamese national legislation—which spans tens of thousands of laws, administrative decrees, and circulars across disjoint regulatory domains—this project deliberately restricts its scope to a representative core of labor law. This intentional scoping serves three main methodological purposes: First, it ensures a highly controlled environment for evaluating the extraction accuracy of legal cross-references and parent-child hierarchical relations, which are critical for graph-based semantic indexing. Second, this controlled scope allows the evaluation to focus on citation coverage, deterministic validation, and reproducible retrieval behavior within the indexed corpus. Third, it directly mirrors the architecture of domain-specific legal expert systems, which prioritize deep, specialized semantic coverage over broad but shallow general knowledge. It is important to emphasize that the proposed system is strictly bounded by this indexed corpus and does not encompass all Vietnamese legislative texts. It is designed as a high-precision, domain-specific advisor for targeted labor-law inquiries and should not be construed as a general legal oracle or a complete substitute for professional legal counsel.

Table 3.1: Summary of the scoped Vietnamese labor law corpus and document metadata.

Document ID	Type	Normative Rank	Chunks
45-2019-qh14	bo_luat	1	697
92-2015-qh13-labor-only	bo_luat	1	159
nghi-dinh-135-2020-nd-cp	nghi_dinh	2	41
nghi-dinh-145-2020-nd-cp	nghi_dinh	2	520
thong-tu-09-2020-tt-bldtbxh	thong_tu	3	73
thong-tu-10-2020-tt-bldtbxh	thong_tu	3	66

A defining characteristic of the Vietnamese legal framework is its strict hierarchical structure of legal validity, which maps directly to the normative ranks defined in Table 3.1. This hierarchical alignment is governed by the national Law on Promulgation of Legislative Documents [2]. Understanding this hierarchy is essential for both query routing and context synthesis, as lower-level regulations must conform to higher-level statutes, and conflicts are legally resolved in favor of the higher-ranking document.

Normative Rank 1 (Codes and Laws). This rank represents the highest level of domestic statutory authority, containing primary legislative acts passed directly by the National Assembly of Vietnam. In our corpus, these are represented by the type **bo_luat** (code). Under the principle of *lex superior*, these documents establish the foundational legal rights, duties, and frameworks that all subordinate instruments must strictly adhere to.

Normative Rank 2 (Decrees). Promulgated by the Government of Vietnam and signed by the Prime Minister, decrees (represented by the type **nghi_dinh**) occupy the second level of the hierarchy. The primary function of a decree is to detail, guide, and implement the broad provisions outlined in a Rank 1 code. While subordinate to codes, decrees carry substantial administrative authority and represent the main operational directives for government ministries and public enforcement.

Normative Rank 3 (Circulars). Positioned at the third level of the hierarchy, circulars (represented by the type `thong_tu`) are issued by individual ministries or ministerial-level agencies. In this corpus, these documents are promulgated by the Ministry of Labour, Invalids and Social Affairs (MOLISA) to provide highly granular, procedural instructions, administrative forms, and narrow technical specifications required to execute decrees and codes. Under Vietnamese legal practice, circulars must remain strictly within the bounds of their parent codes and decrees.

To construct a comprehensive representation of labor relations and disputes, the selected six documents cover the entire span of the legal hierarchy—from primary statutory rights down to specific administrative templates and procedural litigation rules:

Labor Code 2019. Represented by the identifier `45-2019-qh14`, the Labor Code 2019 is the foundational legislative pillar of the entire corpus. Promulgated by the National Assembly on November 20, 2019, and taking effect on January 1, 2021, it establishes the fundamental principles governing labor relations, employment standards, trade unions, social dialogue, and labor dispute resolution. The document comprises 17 chapters and 220 articles, which are preprocessed into 697 distinct text chunks. It acts as the ultimate authority for defining employment contracts, statutory benefits, minimum working age, and occupational rights.

Implementing decrees and circulars. To model the detailed administrative and procedural rules that implement the Labor Code 2019, the corpus integrates four key guiding documents:

- *Decree 145/2020/ND-CP (nghi-dinh-145-2020-nd-cp)*: This Rank 2 administrative decree is a massive regulatory document (yielding 520 chunks) that provides comprehensive guidance on several core aspects of the Labor Code, including specific regulations on labor contract conditions, wage calculations, overtime hours, labor discipline, occupational safety, and mandatory policies for female and minor employees. It represents the primary source of operational details for daily business compliance.
- *Decree 135/2020/ND-CP (nghi-dinh-135-2020-nd-cp)*: This specialized Rank 2 decree establishes the regulatory roadmap for increasing the national retirement age in Vietnam. Crucially, it contains the official retirement age lookup tables and scheduling indexes based on the employee’s gender, birth month, and work hazards, preprocessed into 41 chunks.
- *Circular 09/2020/TT-BLDTBXH (thong-tu-09-2020-tt-bltdtbxh)*: Promulgated by MOLISA, this Rank 3 circular (yielding 73 chunks) details the protective regulations and specific conditions governing employment relations with minor workers under 15 years of age. It includes the official list of permissible light-work occupations and mandatory health-and-safety guidelines for employing youth.
- *Circular 10/2020/TT-BLDTBXH (thong-tu-10-2020-tt-bltdtbxh)*: This Rank 3 circular (yielding 66 chunks) provides detailed templates and mandatory content requirements for labor contracts, covering job scopes, wage structures, social insurance obligations, and intellectual property clauses.

Labor-related civil procedure provisions. Represented by the document ID `92-2015-qh13-labor-only`, this dataset consists of a carefully filtered subset of the Civil Procedure Code 2015 (Law No. 92/2015/QH13), yielding 159 chunks. While the complete Civil Procedure Code governs all civil, commercial, and family lawsuits in Vietnam, our

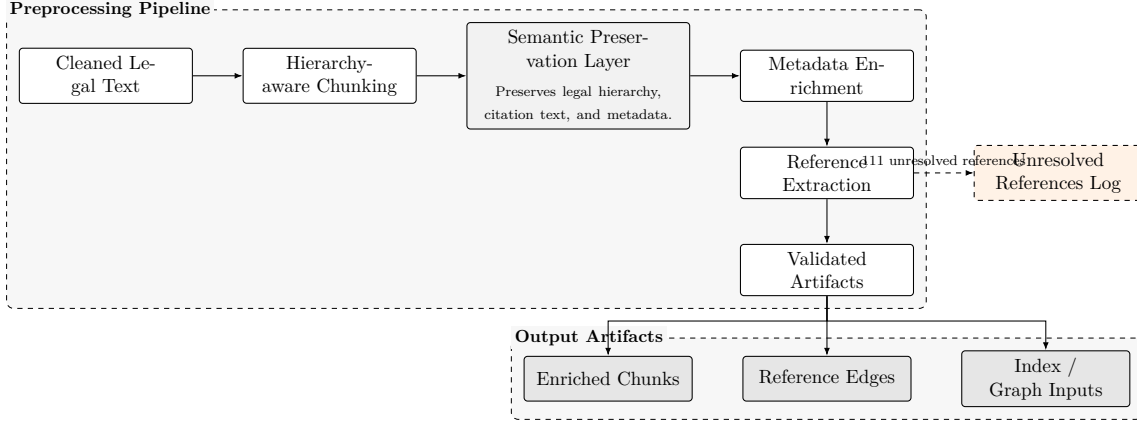


Figure 3.2: Legal data processing pipeline with hierarchy-aware chunking and metadata preservation.

corpus selectively extracts only the specific provisions that govern labor disputes, court jurisdictions, evidentiary requirements, litigation fees, and procedural timelines for labor-related lawsuits. The inclusion of these Rank 1 provisions is vital for query answering because employment-related questions frequently transition from substantive rights (e.g., “What are my rights under unfair dismissal?”) to procedural remedies (e.g., “How and where do I file a lawsuit against my employer?”). By linking litigation procedures directly to substantive labor rights, the corpus enables the QA system to support multi-hop legal reasoning and comprehensive user guidance.

3.3 Legal Document Preprocessing and Enrichment

The performance of a Retrieval-Augmented Generation (RAG) system in the legal domain depends heavily on how raw documents are processed into structured, queryable units. Legal texts use a strict hierarchical organization that is crucial for interpreting statutory provisions. In Vietnam, legislation follows a deeply nested structure of chapters, sections, articles (*Điều*), clauses (*Khoản*), and points (*Điểm*). Standard RAG pipelines typically split documents using flat, fixed-size sliding text windows. However, this arbitrary segmentation is unsuitable for legal texts because it is blind to the structural grammar. It frequently splits cohesive provisions across chunk boundaries, causing semantic fragmentation and separating nested points from their parent clauses and articles. For example, a retrieved chunk containing only “*c) Người từ đủ 15 tuổi đến dưới 18 tuổi*” (c) Persons from full 15 to under 18 years of age) is uninterpretable without knowing this defines the age range for minor employment under Article 143. Naïve chunking destroys the context necessary for dense vector matching and makes it difficult for an LLM to generate or validate precise citations. To address this, our system implements a specialized data processing pipeline (Figure 3.2) that maps raw texts into structured, metadata-enriched chunks.

Text cleaning. Before structural parsing begins, raw documents are converted into normalized Unicode (NFC) encoding to resolve character inconsistencies common in older PDFs. Document-specific headers, footers, page numbers, watermarks, and OCR artifacts are stripped. Double line breaks and indentations are normalized while maintaining paragraph borders, producing a clean, standardized text stream.

Hierarchical chunking. Rather than slicing the text stream at uniform intervals, our chunking engine uses a syntax-aware parser that isolates legal units using deterministic regular expressions. These rules identify key boundaries to map the nested hierarchy into parented, object-oriented structures. The parser detects parts, chapters, sections, articles, clauses, points, and subpoints. The full regular-expression patterns are provided in Appendix B. Crucially, the parser handles subpoint patterns (such as a.1 or b.2) explicitly to prevent granular exceptions or conditions from being merged into generic point texts. This syntax-aware parser processes the document stream to build a lightweight hierarchical representation of nested legal provisions, preserving headings and layout relationships.

Semantic preservation. The core of our preprocessing pipeline is the Semantic Preservation Layer. This layer is designed to maintain the statutory context of nested legal provisions across chunk boundaries by using metadata inheritance. When the hierarchical chunker extracts a granular leaf-level unit, the Semantic Preservation Layer forces that chunk to inherit structural and citation metadata from its parent clause, article, and document. For example, consider the concrete legal provision *Điểm c, Khoản 2, Điều 3* of the Labor Code 2019. Under naive chunking, this point is isolated as a flat string. Under our Semantic Preservation Layer, the resulting chunk explicitly inherits and encapsulates structured key-value parameters such as **point_ref** (“c”), **clause_ref** (“2”), **article_number** (3), **document_id** (“45-2019-qh14”), **document_title** (“Bộ luật Lao động 2019”), and **citation_text** (“Điểm c Khoản 2 Điều 3 Bộ luật Lao động 2019”). This metadata inheritance supports downstream operations by enabling vector searches to be combined with metadata filtering and providing the LLM with the exact legal basis for its statements, making citation generation easier to verify within the indexed corpus.

To optimize both search accuracy and response generation, the pipeline splits each chunk’s textual content into two distinct fields: **retrieval_text** and **citation_text**. The **retrieval_text** field is designed to make semantic vector retrieval more reliable. It combines the raw text of the chunk with parent hierarchical context headers (such as the document title and parent article title). This context injection provides the embedding model with the complete legal framework, helping it calculate more accurate similarity scores even when user queries are implicit or abstract. Conversely, the **citation_text** field represents the exact, formal legal location (e.g., “Điểm c Khoản 2 Điều 3 Bộ luật Lao động 2019”). Splitting these fields is a key design choice: **retrieval_text** supports matching during vector search, while **citation_text** is used for citation grounding and reduces the risk of hallucinations in LLM responses.

Table 3.2: Legal chunking and data integrity validation results.

Validation Item	Value
Legal chunks	1,556
Duplicate chunk IDs	0
Missing citation text	0
Missing article number	0
Very short chunks	0
Very long chunks	0

Metadata enrichment. Beyond structural attributes, the pipeline enriches each chunk with metadata fields to support multi-faceted indexing and advanced query routing,

including operational parameters like `chunk_id`, `chunk_type`, and `source_file`. Crucially, the enrichment pipeline implements a deterministic, rule-based mapping function, `infer_enriched_taxonomy`, to assign legal taxonomy categories without introducing stochastic behavior. The function builds a normalized aggregated context string from the chunk’s hierarchical titles and text, then scans it using rule-based dictionaries containing keyword patterns. Specifically, these dictionaries include `TOPIC_RULES` covering 21 core legal topics, `ACTOR_RULES` covering 10 distinct legal actors, and `ISSUE_TYPE_RULES` covering 15 legal issue types. Using a rule-based dictionary scan rather than stochastic LLM auto-labeling represents a key design choice: it ensures reproducibility, reduces the risk of metadata hallucinations, and reduces ingestion latency and computational costs.

Appendix and table handling. Unlike standard text processors that often struggle to parse non-paragraph elements and discard tables as noise, our pipeline treats appendices and tables as primary structured evidence chunks. This is important because critical legal details—such as the gender-specific retirement age lookup tables in Decree 135/2020/ND-CP or standard contract templates in Circular 10/2020/TT-BLDTBXH—are housed entirely within tables and appendices. Our parser converts these tables into dense markdown representations, preserving column relationships and ensuring that cell-level lookups are searchable. These chunks are embedded and indexed with a dedicated `chunk_type` of `"table"` or `"appendix"` and are linked directly to their parent document nodes in our graph database, allowing the online system to retrieve exact lookup tables.

Validation. To verify the performance of the data processing pipeline before database ingestion, the corpus is passed through an automated validation layer. This layer performs structural sanity checks to detect duplicate identifiers, empty payloads, orphan nodes, and anomalous chunk lengths. Crucially, a quantitative pruning criterion is enforced: any chunks with text shorter than a strict character limit of `VERY_SHORT_TEXT_THRESHOLD = 20` are discarded as fragmented lines or parsing noise, making the validation process quantitative rather than subjective.

As reported in Table 3.2, the preprocessing pipeline of the scoped corpus yielded exactly 1,556 validated legal chunks. The validation run completed with zero duplicate chunk IDs, zero instances of missing citation texts, zero missing article numbers, and zero short or long chunks. This outcome indicates that our pipeline is highly stable: the absence of duplicate chunk IDs shows consistent chunk identification, the zero missing citation text count indicates that every evidence chunk has a valid statutory address, and the lack of very short chunks shows that the parser successfully filtered empty lines and parsing noise below the 20-character threshold without losing genuine legal content.

3.4 Legal Cross-Reference Extraction and Graph Design

Answering legal questions often requires navigating a complex web of cross-references and structural hierarchies. Legal provisions are rarely read in isolation. For instance, a clause in a ministerial circular is frequently qualified by an article in an implementing decree, which in turn derives its authority from a primary code. To capture these structural and thematic relationships, we represent our corpus as a directed legal knowledge graph stored in a Neo4j database. Crucially, this is a deterministic, document-structure legal graph constructed directly from stable legal hierarchies, regular-expression-based citation

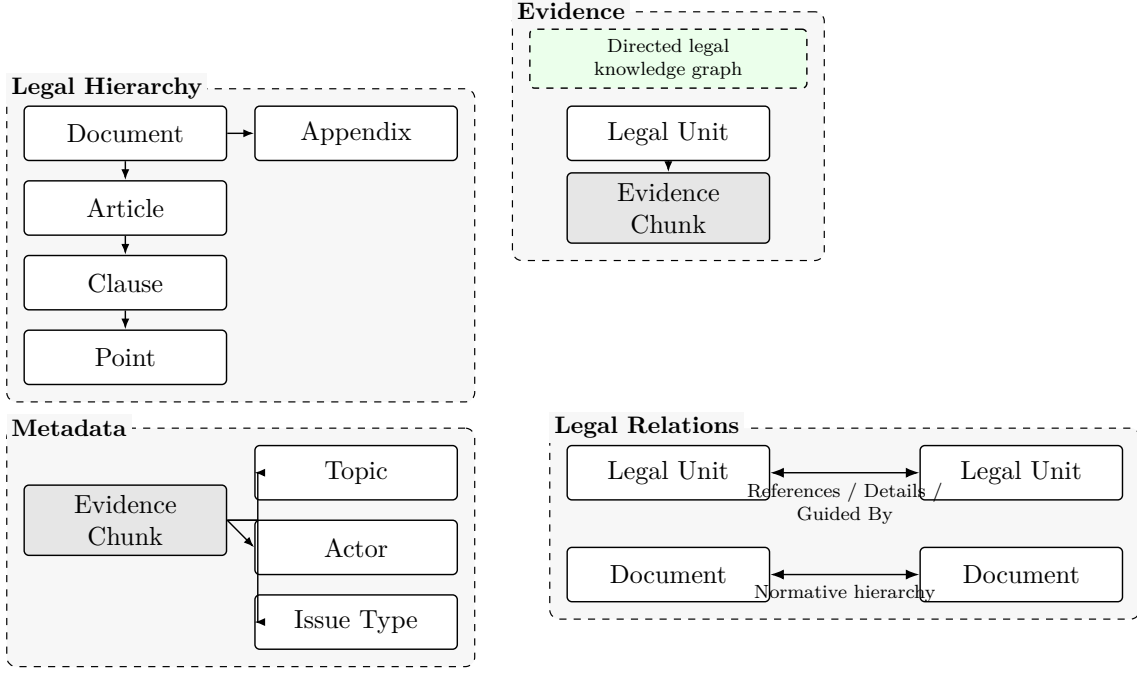


Figure 3.3: Conceptual schema of the Neo4j legal knowledge graph.

patterns, explicit cross-references, rule-based taxonomy metadata, and normative legal validity ranks. It is not a stochastic, LLM-generated entity graph, which would be prone to missing links and hallucinated connections. Using a deterministic schema supports reproducible graph construction and makes the graph easier to inspect.

Graph schema. The conceptual schema of the Neo4j legal knowledge graph is illustrated in Figure 3.3. The graph is organized as a directed network around nine distinct node labels: **Legal_Document**, **Legal_Article**, **Legal_Clause**, **Legal_Point**, **Legal_Appendix**, **Evidence_Chunk**, **Legal_Topic**, **Legal_Actor**, and **Legal_IssueType**. These nodes are connected by relationships that capture the structural, referential, taxonomic, and normative dimensions of the scoped Vietnamese labor-law corpus.

The schema divides relationships into five main functional groups:

1. **Structural hierarchy:** Represents the internal skeleton of each document. Relationships like **HAS_ARTICLE**, **HAS_CLAUSE**, **HAS_POINT**, and **HAS_APPENDIX** connect the structural nodes to map the document nesting directly.
2. **Source chunk/evidence links:** Connects raw **Evidence_Chunk** nodes to their leaf-level parent nodes in the hierarchy (such as points, clauses, or articles) using the **HAS_SOURCE_CHUNK** relationship.
3. **Cross-reference edges:** Model citation links between different legal provisions. Edges such as **REFERENCES**, **DETAILS**, and **GUIDED_BY** enable the retrieval engine to execute graph expansions and gather associated contexts that reside in different parts of the corpus.
4. **Taxonomy edges:** Link **Evidence_Chunk** nodes to semantic classification nodes (**Legal_Topic**, **Legal_Actor**, and **Legal_IssueType**) using relationships such as **MENTIONS_TOPIC**, **APPLIES_TO_ACTOR**, and **HAS_ISSUE_TYPE**. These are used for payload pre-filtering.

Table 3.3: Summary of loaded node and edge counts in the Neo4j legal knowledge graph.

Graph Item	Count
Documents	6
Articles	411
Clauses	1,235
Points	774
Appendices	35
Evidence chunks	1,556
Resolved reference edges loaded	948
Taxonomy edges	6,099
Normative hierarchy edges	20

5. **Normative hierarchy edges:** Represent relationships of legal authority between different documents (e.g., Code $\rightarrow$ Decree $\rightarrow$ Circular) using **SUPERIOR_TO** and **MUST_COMPLY_WITH**. This ensures that higher-rank statutory authority is prioritized.

Citation pattern detection. To build this directed legal network, the preprocessing pipeline uses a specialized citation parsing and reference resolution workflow. The extraction engine scans the raw text of each chunk using regular expressions designed to capture the varied citation styles in Vietnamese legislative documents. The parser identifies absolute citations, which name specific target documents (e.g., “Điều 5 Nghị định số 145/2020/NĐ-CP”), and relative citations, which refer to local or parent structures within the same document (e.g., “Khoản 2 Điều này”). To handle the syntactic complexity of Vietnamese legislative drafting, where multiple clauses are often coordinated in a single sentence, the parser implements a compound reference extractor. For example, a phrase like “khoản 1, 2 và 3 Điều 10” is not collapsed into a single edge; instead, the extractor parses the coordinated clause sequence and programmatically instantiates three separate target references to Clause 1, Clause 2, and Clause 3 of Article 10. This approach preserves graph granularity, ensuring that distinct legal citations can be traversed individually during retrieval.

Reference resolution. Once citation patterns are detected, they are passed to a normalization engine to resolve their target node identifiers. Relative citations are mapped to absolute node paths by looking up the parent article and document ID within the structural hierarchy constructed during preprocessing. Absolute citations are normalized by matching document titles and abbreviations against a registry of aliases (e.g., mapping “Bộ luật Lao động” to the document ID **45-2019-qh14**). Localized relative references like “nghị định này” (this decree), “thông tư này” (this circular), or “luật này” (this law) are resolved dynamically using the source chunk’s metadata context. A notable special case occurs in Article 219 of the Labor Code 2019, which contains transitional and amending provisions. In this article, the expression “Luật này” refers to various external amended laws (such as the Law on Social Insurance) rather than the Labor Code itself. The resolver handles this by mapping the target to a dedicated external amended-law placeholder node instead of forcing it into the internal structure of the Labor Code, which reduces the risk of incorrect edges.

Edge typing. When resolved references are loaded into the database, the edge type between a source node s and a target node t is computed dynamically based on

their legislative normative ranks. Normative ranks are represented as numeric values in our document metadata. In this ranking, Rank 1 represents a primary Code or Law, Rank 2 represents an administrative Decree, and Rank 3 represents a ministerial Circular. Under this scheme, a larger numeric rank denotes a lower-level document in the legislative hierarchy (e.g., a Rank 3 circular is subordinate to a Rank 2 decree, which is subordinate to a Rank 1 code). The formal edge-typing rule is defined by the following mapping:

$$\text{EdgeType}(s, t) = \begin{cases} \text{DETAILS}, & \text{if Rank}(s) > \text{Rank}(t), \\ \text{GUIDED_BY}, & \text{if Rank}(s) < \text{Rank}(t), \\ \text{REFERENCES}, & \text{otherwise.} \end{cases}$$

Under this rule, when a lower-level document cites a higher-level statute (such as a Rank 2 decree detailing a Rank 1 code provision), the edge is typed as **DETAILS**, representing structural elaboration. When a higher-level statute points to a lower-level implementing regulation, the edge is typed as **GUIDED_BY**. Same-rank or neutral references (such as an article in the Labor Code citing another article within the same code) are typed as **REFERENCES**.

Additionally, when a cross-document **DETAILS** edge is generated, the pipeline automatically instantiates a corresponding inverse **GUIDED_BY** edge pointing in the opposite direction. For example, since a decree or circular often details a provision in the Labor Code, the primary **DETAILS** edge captures the citation direction from the lower-level text to the higher-level code. Creating the inverse **GUIDED_BY** edge is a deliberate design choice that allows graph traversal to run in both directions: from implementing guidance back to the source law, and from the source law to its guiding regulations. This bidirectional capability helps graph expansion during retrieval by allowing the traversal engine to gather related documents regardless of which one contains the original citation link.

Unresolved references. To keep the knowledge graph clean, we do not assume all citations can be successfully resolved. Out of 1,059 citation expressions identified by the extraction engine within the scoped corpus, exactly 948 reference edges were resolved and loaded into the Neo4j knowledge graph, as summarized in Table 3.3. The remaining 111 references were logged and intentionally excluded from graph traversal, rather than being treated as system errors. This deliberate omission represents a conservative graph construction philosophy that keeps uncertain references out of the active graph. The 111 unresolved references primarily stem from references to external legislative documents outside our six-document corpus (such as the Civil Code 2015), incomplete textual coordinates, references to external laws, and ambiguous relative references that lacked specific anchor points. Excluding these uncertain references keeps the active graph conservative, but it also limits graph expansion to references that can be resolved within the indexed corpus.

3.5 Vector-based and Hybrid Retrieval

Before executing graph-based context expansion, the system establishes a baseline retrieval layer using vector-based and hybrid search. Relying on a single retrieval modality is often insufficient for legal question answering. While dense semantic search is highly effective at identifying conceptual synonyms, it can miss or be less effective for matching exact statutory citations or numerical coordinates. Conversely, sparse keyword matching is very precise at capturing exact article numbers and legislative IDs but is less effective for

matching paraphrased conversational queries. To resolve these individual limitations, our system implements a unified hybrid retrieval framework that combines both approaches using the Qdrant vector database [5] to index and query our preprocessed legal corpus.

The baseline retrieval engine operates over the 1,556 hierarchy-aware legal chunks in our scoped corpus. During database ingestion, each chunk is encoded into both dense and sparse vector representations. The dense semantic embeddings are generated using the **keepitreal/vietnamese-sbert** model [6], which is a Sentence-BERT transformer optimized for Vietnamese natural language processing. This model projects each text chunk into a 384-dimensional dense vector space. For sparse lexical matching, the chunk text is processed using a sparse encoder to generate term-importance vectors that capture structural keywords and statutory terminology. Crucially, each vector index entry in Qdrant preserves a comprehensive payload of metadata to enable advanced pre-filtering and query routing. This metadata payload includes the following ten fields: **chunk_id**, **document_id**, **article_number**, **clause_ref**, **citation_text**, **document_type**, **normative_rank**, **topic**, **actor**, and **issue_type**. Preserving this payload allows the system to apply legal intent-specific constraints during query execution.

At query execution time, the search engine does not simply calculate the similarity score of the raw user query. Instead, as shown in our overall pipeline, the query first passes through the routing layer to identify structural intents, documents, and coordinates. The system then generates multiple semantic query variations using the internal helper **build_query_variants**. This step helps bridge the lexical gap between a user’s informal conversational question and the highly formal statutory phrasing of Vietnamese labor regulations. The generated set of query variants, V , can include the raw user query, rule-based query expansions, issue-focused terms, and specific citation coordinates. For each query variant, the retrieval engine queries Qdrant using parallel prefetch channels that run both dense semantic and sparse keyword searches concurrently.

To reduce embedding misalignment and avoid relying on a single vector representation of the user query, the system prefetches candidates across multiple dense and sparse channels. The total number of parallel prefetch channels, $|P|$, executed for a single query is determined by the following relation:

$$|P| = 2|V| + \mathbb{I}(\text{ref_boost}) + \mathbb{I}(\text{issue_focus})$$

where V is the set of generated query variants, and P is the set of Qdrant prefetch channels. The term $2|V|$ represents the concurrent execution of one dense and one sparse prefetch channel for each generated query variant. The term $\mathbb{I}(\text{ref_boost})$ is an indicator function that equals 1 when a reference-boost filter is active, and 0 otherwise. Similarly, $\mathbb{I}(\text{issue_focus})$ is an indicator function that equals 1 when an issue-focus filter is active, and 0 otherwise.

In the implemented retrieval engine, both the reference-boost filter (**ref_boost**) and the issue-focus filter (**issue_focus**) are routed exclusively through the sparse vector prefetch channel. This is an important design choice: because article numbers, document IDs, issue types, and legal actors are represented as deterministic legal tokens, they are matched more reliably by sparse lexical indexing combined with payload filtering. Routing these high-precision boosts through the sparse vector channel rather than relying on dense embeddings helps reduce dense semantic noise and prevents exact legal coordinates from being blurred by superficial embedding similarities.

Once candidate lists are returned from all active prefetch channels, the search engine

merges and reranks them using Reciprocal Rank Fusion (RRF). In our implementation, this fusion operation is delegated directly to Qdrant’s built-in RRF operator. Conceptually, the RRF score for a candidate chunk d is computed by the following mapping:

$$\text{RRF}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

where M represents the set of active prefetch channels, $r_m(d)$ is the ordinal rank of candidate d within prefetch channel m (with $r_m(d) = \infty$ if d is absent from the retrieval list of channel m), and k is a smoothing constant that helps prevent top-ranked items in any single channel from dominating the fused result. Because this fusion is handled natively by Qdrant’s query engine, the formula above represents the underlying ranking principle, while the exact value of the smoothing constant k is managed by the backend search engine configuration.

Fusing dense semantic search with sparse lexical precision improves the chance of retrieving highly relevant chunks (such as exact articles like “Điều 219”, or documents like “Nghị định 145” and “Thông tư 10”) while maintaining broad semantic coverage. However, while hybrid retrieval provides a strong baseline retrieval layer, it still treats chunks as isolated evidence and remains oblivious to the structural and referential bonds between them. For example, if a primary code article (such as Article 143 on minor workers) explicitly references a ministerial circular for detailed occupational lists (such as Circular 09/2020/TT-BLDTBXH), a hybrid search can retrieve the Labor Code chunk but cannot dynamically traverse the relationship to fetch the circular’s details unless the query itself explicitly contains those circular-specific terms. To answer complex multi-hop queries, the system must navigate these statutory cross-references. In our architecture, the hybrid retrieval layer is used to provide a set of strong seed chunks for graph expansion.

3.6 Query Routing and Graph Expansion Policy

As discussed in Section 3.5, while hybrid search provides strong baseline seed chunks S_k , resolving complex legal questions requires traversing statutory cross-references, taxonomic connections, and legislative hierarchies. In our implementation, the query routing layer acts as the main routing component. It translates a raw, conversational user query into a structured, formal **QueryIntent** control object. This control object dynamically guides subsequent retrieval stages, including candidate seed pre-filtering, forced reference promotion, coordinate fallback, and adaptive context windowing. These steps enrich the retrieval context before generation. Specifically, query routing resolves the legal intent and constraints, which enables graph-based reference expansion to fill the missing legal context.

Intent resolution. The query routing layer executes a dual-stage routing process consisting of a structured, language-model-based metadata extractor to resolve semantic intent, and a deterministic validation and expansion layer to align queries with the statutory domain. To extract structural intent, a structured language model acts as a metadata extractor. This model structures the query’s legal context into a schema containing fields such as **actors**, **topics**, **issues**, **document_ids**, **query_types**, **article_numbers**, **clause_refs**, and **point_refs**. Importantly, the model is configured with temperature set strictly to 0 to minimize stochastic variance. The model is used only for metadata extraction, not for generating legal answers, which reduces the risk of generating inaccurate or hallucinated statutory text.

Metadata sanitization. Once the structured metadata payload is returned, the system routes it through a cleaning step that normalizes and filters all extracted metadata against valid domain vocabularies, such as valid actors, topics, issues, document IDs, and query types. This sanitization layer reduces invalid metadata by discarding out-of-vocabulary terms. Formally, for any extracted metadata collection, the sanitization operation resolves valid labels through the following mapping:

$$L_{\text{valid}} = \{ l \in L_{\text{extracted}} \mid \text{Normalize}(l) \in \Omega \}$$

where $L_{\text{extracted}}$ is the set of raw labels extracted by the language model, Ω represents the corresponding set of valid domain labels, and the function $\text{Normalize}(l)$ represents a deterministic unicode and casing normalization pipeline. Casing is standardized to lowercase, accents are stripped, and spaces are mapped to underscores. Discarding invalid labels prevents stochastic errors from entering the retrieval pipeline.

QueryIntent construction. After the metadata is sanitized, the system constructs a unified **QueryIntent** control object. Rather than relying solely on the language model’s classification, the system dynamically merges the sanitized metadata with keyword matches extracted directly from the normalized query stream, rule-based routing coordinates, and mapped article coordinate expansions. The resulting **QueryIntent** represents the central control object that links query understanding to retrieval execution. It contains all active actor, topic, issue, and document filters, alongside explicit and inferred article numbers, ensuring that subsequent retrieval layers are guided by a unified, structured intent.

Forced reference promotion. When a user query matches high-precision legal rules, the routing engine identifies specific statutory provisions as mandatory contexts that must be retrieved, regardless of their vector similarity score. These are represented as high-confidence forced references inside the set $\mathcal{R}_{\text{forced}}$. To prioritize these legally required documents, the system fetches them directly by their coordinates and promotes them. The system uses a deterministic score adjustment for each promoted forced-reference chunk d_{forced} using the following formula:

$$\text{Score}(d_{\text{forced}}) = \max_{h \in S_k} \text{Score}(h) + \Delta_{\text{forced}} - \rho \cdot 10^{-4}$$

where S_k is the set of seed chunks returned by the hybrid vector search, $\max_{h \in S_k} \text{Score}(h)$ is the maximum relevance score among those baseline seeds, Δ_{forced} is the forced-reference score margin (configured as a positive constant), and ρ represents the ordinal rank of the forced record. This score adjustment ensures that every mandatory forced-reference chunk receives a score higher than the highest-scoring vector search hit, helping prioritize high-confidence legal coordinates.

Coordinate fallback. Beyond explicit forced references, the system adds a fallback path to retrieve specific statutory articles that the baseline hybrid search might omit due to vector space limitations. The system compares the required article coordinates in the query intent (A_{required}) against the article numbers actually present in the retrieved hybrid search hits ($A_{\text{retrieved}}$). The set of missing target articles, A_{missing} , is computed via:

$$A_{\text{missing}} = A_{\text{required}} \setminus A_{\text{retrieved}}.$$

If A_{missing} is non-empty, the system fetches the missing statutory chunks directly from the record store. If the fallback is triggered by an explicit mention of an article number

in the user’s query (explicit fallback), the fetched chunks are promoted slightly above the current hits, receiving a score boost of $+10^{-3}$ relative to the maximum existing score. If the fallback is triggered by rules that infer article numbers from topic/issue mappings (inferred fallback), the chunks receive a minor score penalty of -10^{-3} relative to the maximum score. The explicit fallback receives a stronger score adjustment, while the inferred fallback is kept more conservative. This fallback mechanism reduces the risk of missing explicit legal coordinates.

Adaptive context window. To balance retrieval coverage and prompt noise, the system implements an adaptive context windowing policy. When a query involves highly complex legal issues or comparative intent, a narrow context window may truncate critical legislative provisions, whereas a broad window for simple lookups introduces unnecessary noise that degrades generation quality. To resolve this, the system dynamically scales the primary article retrieval limit, L_{primary} , using the following adaptive relation:

$$L_{\text{primary}} = \begin{cases} 8, & \text{if } T_q \cap \Theta_{\text{complex}} \neq \emptyset \text{ or } I_q \cap \Phi_{\text{complex}} \neq \emptyset, \\ 4, & \text{otherwise.} \end{cases}$$

where T_q is the set of query types extracted in the intent, I_q is the set of active issue filters, and Θ_{complex} and Φ_{complex} represent the configured sets of complex query types and complex issues. For simple queries, the limit defaults to 4 articles, while secondary article limits are kept smaller (typically 2 to 4). This adaptive limit controls prompt noise while ensuring sufficient coverage for complex queries.

Final context assembly. Because the seed chunks S_k from the hybrid retrieval layer share identical coordinates with graph nodes, they are directly mapped into Neo4j graph nodes for traversal. The system then traverses the knowledge graph to gather topologically associated legal provisions—such as parent-child structural units, cross-document circular details, or normative hierarchy links. The final retrieved context set C_{final} is consolidated by merging the baseline seed chunks S_k , the graph-expanded context C_{graph} , and the coordinate fallback chunks C_{fallback} :

$$C_{\text{final}} = \text{Rerank}(S_k \cup C_{\text{graph}} \cup C_{\text{fallback}}).$$

This final context set is passed to the grounded answer generator, where evidence selection, reranking, and citation validation are executed, as described next in Section 3.7.

3.7 Grounded Answer Generation with Legal Citations

The final stage of the pipeline converts the consolidated, reranked retrieval contexts into a citation-grounded answer (Figure 3.5). Directly presenting raw language model outputs in professional legal settings can introduce risks such as semantic hallucinations and citation misattributions. To reduce the risk of unsupported claims, the answering layer runs retrieved contexts through semantic reranking, normative-rank context budgeting, and a deterministic validation gate. The system supports structured answer generation, but final quality evaluation relies on deterministic rule-based validation. Notably, answer quality is evaluated using deterministic rule-based validation, not human legal expert review or LLM-as-Judge evaluation.

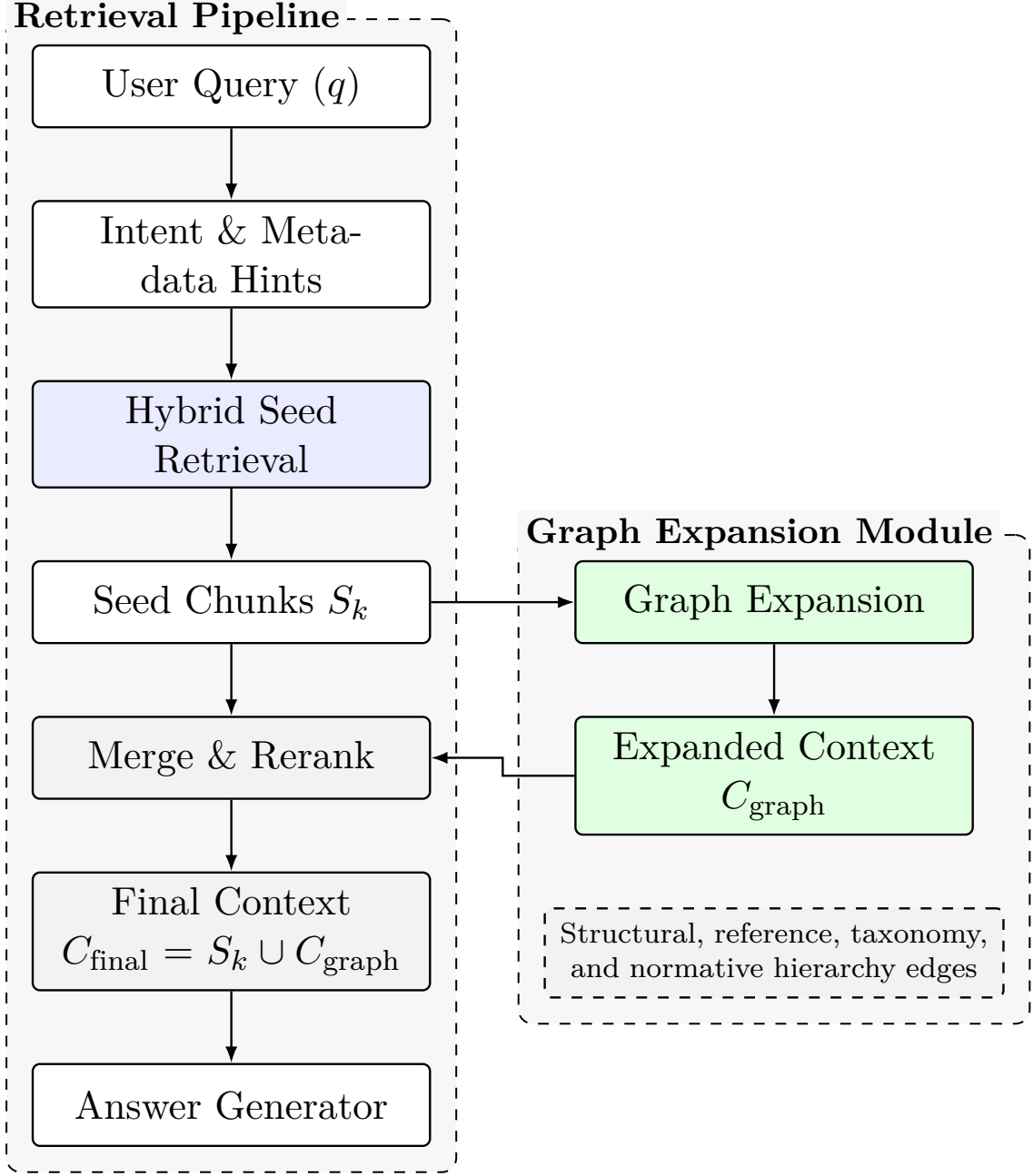


Figure 3.4: Graph-augmented retrieval flow combining hybrid seed retrieval and legal graph expansion.

Semantic reranking. To improve context relevance, the pooled context candidates are processed by a semantic cross-encoder model. While standard vector search is effective for broad initial retrieval, a cross-attention transformer provides finer granularity by analyzing attention weights across the query-context pairs. Candidate chunks within a parameterized limit ($N = 24$) are evaluated against the query q using a cross-encoder model. To preserve the initial search hierarchy, the system does not discard the initial search order; instead, it computes a Reciprocal Rank Fusion (RRF) score by combining the chunk’s heuristic retrieval rank $R_{\text{heuristic}}(d)$ with its semantic cross-encoder rank $R_{\text{semantic}}(d)$:

$$\text{RRF}_{\text{fused}}(d) = \frac{1}{K_{\text{RRF}} + R_{\text{heuristic}}(d)} + \frac{1}{K_{\text{RRF}} + R_{\text{semantic}}(d)}$$

where the rank-stabilization constant K_{RRF} is set to 60. Chunks are sorted in descending order of their RRF scores to prioritize the most semantically relevant evidence.

Normative-rank context budgeting. Following semantic reranking, the system executes a context budgeting algorithm to select the evidence window. Statutory provisions in Vietnamese labor law carry different levels of authority: Rank 1 represents primary Codes (**bo_luat**), Rank 2 denotes administrative Decrees (**ngghi_dinh**), and Rank 3 corresponds to ministerial Circulars (**thong_tu**). To prevent detailed implementing guidelines from overshadowing primary statutory provisions, the system enforces a strict budget limit $B(r)$ based on document rank r :

$$B(r) = \begin{cases} 3, & r = 1 \text{ (Codes/Laws)}, \\ 2, & r = 2 \text{ (Decrees)}, \\ 2, & r = 3 \text{ (Circulars)}. \end{cases}$$

Additionally, contexts are dynamically deduplicated using a unique article key composed of the document identifier and article number. This prevents multiple chunks from a single article from consuming the budget, encouraging document diversity in the final prompt context.

Citation-aware answer construction. Grounded answers are generated from the budgeted contexts using structured prompt formats. The answering layer can force a JSON schema to parse the model output into specific fields: the natural language **answer**, the canonical citations as **legal_basis**, direct source text pairings as **evidence_quotes**, an **insufficient_context** boolean flag, and additional **notes**. To maintain a consistent structure, the answer starts with a lead sentence generated by a semantic anchoring transition (Γ), which maps user query patterns to direct legal transitions:

$$\text{Lead}_{\text{answer}} = \Gamma(q, c_1),$$

where q is the query and c_1 is the highest-ranked context chunk. This structuring guides the output to flow logically from primary laws to secondary implementing regulations, using only retrieved contexts and the designated legal basis while avoiding unsupported article numbers.

Deterministic validation gate. Before the generated answer is returned, a validation gate evaluates the output to check whether citations are supported. The validation check computes the logical conjunction of five rule-based guardrails:

$$\Phi_{\text{validation}} = \phi_{\text{citation}} \wedge \phi_{\text{retrieved}} \wedge \phi_{\text{article}} \wedge \phi_{\text{rank}} \wedge \phi_{\text{uncertainty}}.$$

These individual validation guardrails check whether citations are supported within the retrieved corpus:

- **Inline Citation Guard** (ϕ_{citation}): Confirms that the answer text contains inline citations pointing to the legal basis, unless the system has flagged that context is insufficient:

$$\phi_{\text{citation}} = \mathbb{I}(\text{Answer contains citation}) \vee \mathbb{I}(\text{insufficient_context} = \text{true})$$

- **Retrieval Boundary Guard** ($\phi_{\text{retrieved}}$): Checks whether all citations listed in the legal basis are present within the retrieved context space ($\mathcal{C}_{\text{retrieved}}$), blocking any out-of-bounds references:

$$\phi_{\text{retrieved}} = \mathbb{I}(\text{Citations} \in \mathcal{C}_{\text{retrieved}})$$

- **Article Numeral Verification** (ϕ_{article}): Verifies that any article numbers mentioned in the answer text (A_{answer}) or the legal basis (A_{basis}) are a subset within the articles of the retrieved contexts (A_{contexts}):

$$A_{\text{answer}} \cup A_{\text{basis}} \subseteq A_{\text{contexts}}$$

- **Normative Rank Priority Gate** (ϕ_{rank}): Checks that the answer does not cite low-rank circulars while ignoring higher-authority codes present in the retrieved contexts. It requires $r_{\text{cited}}^{\min} \leq r_{\text{available}}^{\min}$, where $r^{\min}$ denotes the highest authority level (minimum numeric value):

$$\phi_{\text{rank}} = \mathbb{I}(r_{\text{cited}}^{\min} \leq r_{\text{available}}^{\min})$$

- **Uncertainty Disclosure Gate** ($\phi_{\text{uncertainty}}$): Confirms that if the context is insufficient, the text includes standard uncertainty indicators to signal limits in evidence coverage:

$$\phi_{\text{uncertainty}} = \mathbb{I}(\text{Answer contains uncertainty}) \vee \mathbb{I}(\text{insufficient_context} = \text{false})$$

If any guardrail fails ($\Phi_{\text{validation}} = 0$), the stochastically generated answer is flagged as invalid, and the system uses fallback mechanisms.

Extractive fallback. When the structured generator fails to produce a valid response format or flags invalid outputs via the validation gate, the system triggers a deterministic extractive fallback. This fallback mechanism bypasses language model generation entirely. By leveraging the structural metadata from the preprocessing stage, the pipeline extracts canonical paragraph snippets (**_best_rule_snippet**) directly from the retrieved context chunks. It filters out low-information lines using a minimum token threshold and concatenates the remaining high-relevance snippets, anchoring each statement to its native citation. This approach ensures that when text generation fails, the system still provides a reliable, citation-supported answer constructed entirely from the retrieved source text.

Interpretation and limitations. The rule-based validation gate and extractive fallback are designed to improve reproducibility and minimize hallucinated citations within the retrieved corpus. However, this benchmark-specific validation is deterministic and rule-based; it does not replace professional legal expert review, nor does it guarantee universal legal correctness of the generated content. Instead, it serves as a software-level verification step to ensure the output aligns with the retrieved documents.

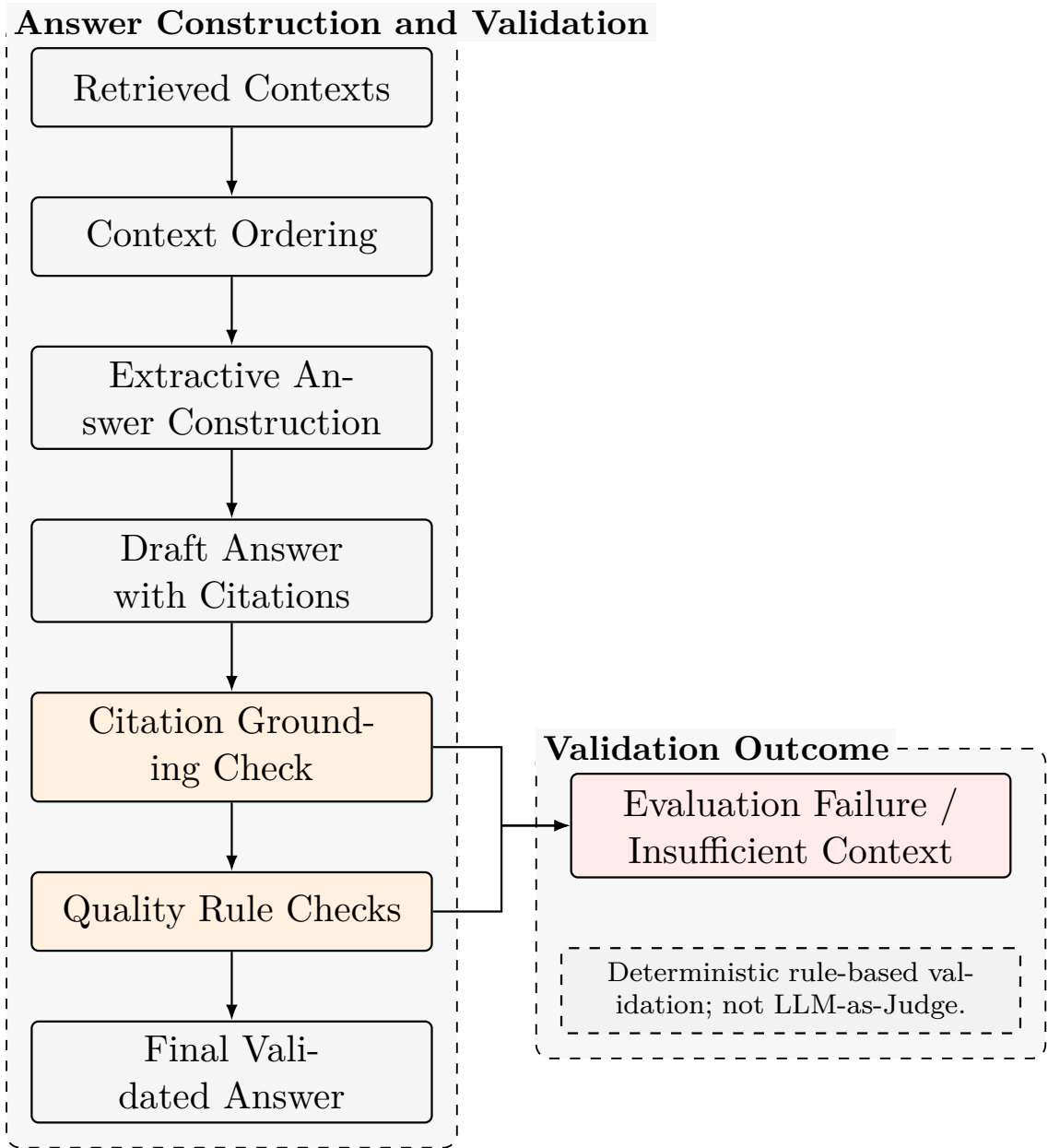


Figure 3.5: Answer generation and deterministic citation validation flow.

Chapter 4

Implementation

4.1 Technology Stack

The implemented legal question-answering system is constructed as a modular Python-based pipeline designed for high performance, strict citation safety, and reproducible evaluations. The backend orchestration layer is written in Python (version 3.10+) and exposes high-level service interfaces via FastAPI. Data persistence, semantic indexing, and relational traversal are partitioned across two specialized database layers. For vector-based and keyword-based search, the system integrates a Qdrant vector database collection, which maintains dense high-dimensional semantic encodings and sparse index structures. For relational cross-reference modeling and topological query expansion, the system utilizes a Neo4j graph database instance configured with the APOC (Awesome Procedures on Cypher) plugin library.

Table 4.1: Technology stack.

Layer	Technology	Purpose
Backend	Python, FastAPI	API and orchestration
Embeddings	Sentence Transformers	Dense multilingual vectors
Vector database	Qdrant	Dense and sparse retrieval
Graph database	Neo4j	Legal hierarchy and references
Evaluation	Python scripts	Benchmark reporting

The dense embeddings are generated locally using the Sentence Transformers framework, employing the pre-trained **keepitreal/vietnamese-sbert** model (yielding 384-dimensional vectors). To ensure high reproducibility, the database dependencies—specifically the Neo4j Community Edition 5.26 service—are containerized using Docker and managed programmatically via Docker Compose (using the **docker-compose.neo4j.yml** configuration). Global execution parameters, including connection URIs (**QDRANT_URL**, **NEO4J_URI**), authentication credentials (**NEO4J_AUTH**), active models (**DENSE_MODEL**), and graph traversal depths, are fully parameterized through a central **.env** environment file.

4.2 Data Processing Implementation

The transformation of raw legal texts into validated, queryable vector and graph structures follows a strict pipeline of Python scripts located in the **scripts/** directory. The pipeline executes in the following sequential order:

1. **Clean Text Validation:** The raw Vietnamese labor law documents and circulars are validated for syntax correctness, header structure, and encoding issues using the **validate_curated_legal_texts.py** script. The results are logged as validation reports in **artifacts/validation/**.
2. **Chunk Generation:** The script **build_legal_chunks.py** parses the validated, cleaned legal texts in **corpus/cleaned/** into hierarchical, context-aware units. The script analyzes document structural patterns (such as chapters, articles, clauses, and points) and writes the results to **artifacts/chunks/legal_chunks.jsonl** (and generates **legal_chunks_summary.md**), producing 1,556 chunks.
3. **Metadata Enrichment:** The script **enrich_legal_chunks.py** maps semantic attributes to each chunk, including general topics, specific actors, issue types, document types, and legislative normative ranks. It writes the enriched output directly to **legal_chunks_enriched.jsonl**.
4. **Cross-Reference Extraction:** Relative and absolute legal cross-references within each chunk are parsed and extracted by running **build_reference_edges.py**. This generates a raw reference-edge list in **artifacts/graph/reference_edges.jsonl**. Unresolved citations are automatically flagged and written to a diagnostic report via **analyze_unresolved_reference_edges.py**.

4.3 Vector Index and Hybrid Retrieval Implementation

The vector indexing layer ingests the 1,556 enriched legal chunks and populates the Qdrant vector store. This process is orchestrated by running the script **build_index.py** with the **--collection-name vietnamese_labor_law_chunks** flag. During indexing, each chunk’s textual content is converted into a 384-dimensional dense vector using the configured multilingual embedding model.

The resulting vector points are uploaded to Qdrant alongside a structured payload dictionary. This payload stores extensive metadata fields—such as the chunk ID, primary document ID, article number, clause reference, normative rank, topic labels, and the original Vietnamese citation text. Hybrid retrieval fuses dense vector similarity searches (capturing semantic proximity and synonym variations) with sparse token matching (ensuring that specific article numbers, statutory coordinates, and keywords are matched exactly). The payload fields are utilized during retrieval to enforce runtime filters (such as topic or document exclusions), support downstream citation validation, and enable deterministic context reranking before prompting.

4.4 Neo4j Graph Implementation

To construct the relational backbone of the corpus, the system deploys a local Neo4j 5.26 Community Edition container via the `docker-compose.neo4j.yml` compose file. The legal graph is loaded and validated using the `build_legal_graph.py` script, which connects to the database via the Bolt protocol on port 7687 using credentials configured in the `.env` file (`NEO4J_USER=neo4j`, `NEO4J_PASSWORD=password`).

The graph model defines node types representing the legislative hierarchy, including *Document*, *Article*, *Clause*, *Point*, *Appendix*, and *EvidenceChunk*, as well as semantic nodes like *Topic*, *Actor*, and *IssueType*. Relationships are loaded from the processed reference-edge artifacts, including structural links (establishing parent-child hierarchies between Articles, Clauses, and Points) and taxonomy edges. Legal cross-references are represented using specific edge types: **REFERENCES** (general statutory citations), **DETAILS** (provisions that provide specific definitions), and **GUIDED_BY** (links establishing administrative delegation between a parent code and executive decrees or circulars). Unique constraints and indexes are applied to the `id` properties of all nodes to ensure upsert integrity. In this build, 111 unresolved references were flagged and excluded from the Neo4j instance to prevent traversing dead-end or invalid citation paths.

4.5 Graph-Augmented Retriever Implementation

The graph-augmented retriever is implemented in the `vn_labor_law_ai_assistant/rag/` module. The execution flow coordinates Qdrant searches and Neo4j graph queries to establish a complete legal context window:

1. **Seed Retrieval:** The query is routed to Qdrant, where hybrid retrieval fetches the top- k (typically 10) seed chunks.
2. **Node Mapping:** The retriever extracts the unique `chunk_id` values from the seed results and maps them directly to the matching *EvidenceChunk* nodes in the Neo4j database.
3. **Topological Expansion:** Starting from the mapped seed nodes, the retriever executes a parameterized Cypher query using the Bolt driver to traverse reference edges (**DETAILS**, **GUIDED_BY**, **REFERENCES**), taxonomy links, and structural parents. The expansion depth is controlled via the `LEGAL_GRAPH_EXPANSION_DEPTH` variable (typically set to 2).
4. **Context Assembly:** The retriever merges the newly discovered graph chunks with the original seeds, resolves score weights based on topological distance, deduplicates duplicate records by chunk ID, and selects the top-ranked contexts up to the maximum prompt limit (typically 8 to 12 chunks) for downstream generation.

4.6 Grounded Answer Generator and Validation

The grounded answer generation layer is implemented in the `vn_labor_law_ai_assistant/answering.py` and `rag/answering/` modules. When LLM provider access is enabled through environment variables (such as `LLM_PROVIDER=groq`), the system constructs a system prompt containing the assembled context block and a structured JSON schema

(`ANSWER_JSON_SCHEMA`). The LLM is instructed to output a parsed JSON payload containing the natural language answer, specific parenthetical citation labels, and exact evidence quotes from the retrieved text.

To ensure safety and prevent hallucinations, the generation layer runs the response through a deterministic validation pipeline. The validation checks enforce three main rules: (1) every generated citation must map exactly to an article present in the retrieved context; (2) every statement of legal basis must match the original Vietnamese wording; and (3) no unsupported article numbers are allowed. If the validation check fails, or if the LLM provider is disabled, the system automatically triggers an extractive fallback using the `build_extractive_answer_payload` function. This fallback directly extracts the validated legal provisions from the retrieved chunks and formats them as the final output. This fallback is used to reduce unsupported generation and ensure strict compliance with the indexed corpus.

4.7 Evaluation Scripts and Reproducibility

The evaluation pipeline contains scripts designed to measure retrieval performance and end-to-end QA safety over the 69-query benchmark. The evaluations are fully reproducible using the commands detailed in `artifacts/evaluation/reproducibility_notes.md`:

- **Retrieval-Mode Evaluation:** Run using the `scripts/evaluate_retrieval_modes.py` script. This script compares vector-only, hybrid, and graph-augmented configurations on the diagnostic benchmark, writing results (Recall@10, required citation coverage, and forbidden citation violation rates) to `artifacts/evaluation/retrieval_modes_expanded_results.csv`.
- **End-to-End Evaluation:** Executed via `scripts/evaluate_end_to_end_rag.py`. This script runs the full retrieval-to-generation pipeline, passing outputs through the validation gates. The script writes detailed JSON traces and CSV summaries directly to `artifacts/evaluation/end_to_end_expanded_results.json` and `end_to_end_expanded_report.md`.
- **Benchmark Coordination:** Overall execution and diagnostic checks are managed by the `scripts/run_benchmark.py` coordinator.

4.8 Example Use Cases

The utility of the implementation is demonstrated across several common legal inquiry patterns evaluated on the diagnostic benchmark:

- **Minor Worker Employment:** A query regarding hiring a 14-year-old worker (`LBR_079`) triggers seed retrieval of BLLĐ Article 143, followed by a graph-augmented traversal to Circular 09/2020/TT-BLĐTBXH, which extracts the exact administrative list of allowed occupations.
- **Retirement Age Table Lookup:** An inquiry about female retirement ages in 2026 (`retirement_female_2026`) is routed to BLLĐ Article 169, which is linked via a `DETAILS` edge directly to Appendix I of Decree 135/2020/NĐ-CP, allowing the retriever to capture the exact tabular row and output 57 years.

- **Labor Contract Content:** A question asking for mandatory contract details (**contract_content_guidance**) retrieves Article 21, then traverses the graph to Circular 10/2020/TT-BLDTBXH Article 3 to pull exact administrative guidelines.
- **Dismissal Dispute Procedure:** A multi-hop query about dismissal mediation (**guidance_dismissal_dispute**) connects BLLĐ Article 188 to Article 32 of the Civil Procedure Code (BLTTDS 2015), linking substantive rights and court jurisdictions.

Chapter 5

Evaluation and Results

5.1 Evaluation Setup and Benchmark Design

This section describes the empirical evaluation setup, the constructed query benchmark, and the corpus indexing details. The system is evaluated on a constructed benchmark consisting of 69 queries, designed to measure how well the pipeline performs retrieval, citation tracking, and end-to-end answering within the indexed six-document Vietnamese labor-law corpus. To evaluate statutory correctness and citation consistency, answer quality is assessed using deterministic rule-based validation within this bounded corpus, rather than relying on human legal expert review or LLM-as-Judge evaluation. This evaluation does not prove universal legal correctness of the system’s output across all legal contexts, but serves as a software-level verification on the constructed benchmark.

Corpus and index statistics. The primary source material comprises six Vietnamese labor-law documents. During document preprocessing, these documents were parsed and divided into 1,556 structural chunks. For the vector retrieval layer, these chunks are stored as 1,556 dense vector points in a Qdrant collection. For the graph retrieval layer, a directed knowledge graph is instantiated in Neo4j to capture the relationships between various legal elements. The graph contains structured nodes for documents, articles, clauses, points, and appendices, alongside the 1,556 evidence chunks. To ensure search paths remain valid during graph traversal, only successfully resolved cross-references are loaded as edges, while unresolved citations are excluded from graph traversals. Table 5.1 details the structural distribution and edge counts loaded into the active system.

Table 5.1: Corpus and index statistics.

Item	Value
Corpus documents	6
Legal chunks	1,556
Vector index count	1,556
Neo4j documents	6
Neo4j articles	411
Neo4j clauses	1,235
Neo4j points	774
Neo4j appendices	35
Neo4j evidence chunks	1,556
Loaded REFERENCES Edges	290
Loaded DETAILS Reference Edges	329
Loaded GUIDED_BY Reference Edges	329
Loaded Taxonomy Metadata Edges	6,099
Loaded Normative Hierarchy Edges	20
Logged Unresolved References Excluded from Graph Traversal	111

Benchmark construction. To evaluate the RAG pipeline under different query conditions, we constructed a diagnostic benchmark of 69 queries. These queries are distributed across nine structural categories to reflect typical questions about labor regulations: Direct QA (19), Definition QA (7), Procedure QA (7), Scenario-based QA (4), Document guidance QA (10), Comparison QA (4), Exception-based QA (11), Multi-hop QA (2), and Calculation or table lookup (5).

To test retrieval capabilities under varying levels of lexical and structural complexity, the queries are classified by difficulty: 15 easy queries, 32 medium queries, and 22 hard queries. Easy queries typically feature direct phrasing and simple vocabulary that align closely with the source text. Medium queries introduce mild vocabulary variations or require matching multiple clauses within the same document. Hard queries involve multi-hop relationships, complex procedural sequences, or table lookups where shallow vector similarity alone is often insufficient. Within this benchmark, 25 queries are annotated as graph-required, meaning they are designed to benefit from graph-based expansion through cross-reference, structural, taxonomy, or normative-hierarchy relations. Additionally, 22 queries are annotated as normative-hierarchy-required to evaluate transitions between statutory tiers, such as moving from a primary Code down to implementing Decrees. Each query is annotated with a ground-truth set of required citations needed to support a correct answer, and some queries include optional forbidden citations representing obsolete or irrelevant clauses that the retriever should avoid.

Evaluation execution environment. The evaluation was carried out in a local environment using a Python virtual environment. UTF-8 console configurations were used to support Vietnamese text without character corruption. The backend database layer consists of a local Qdrant collection storing 384-dimensional dense vectors generated by the `keepitreal/vietnamese-sbert` embedding model, alongside a local Neo4j graph instance for relational lookup. Retrieval comparison experiments evaluate candidate lists with a top- $k = 10$ context constraint for Recall@10. The end-to-end evaluation integrates the retrieved candidate contexts with an extractive answer generator to produce citation-grounded output. The resulting answers are validated using deterministic, rule-based

validation checks to verify correctness against the benchmark annotations.

The remainder of this chapter is organized as follows. Section 5.2 defines the retrieval evaluation methodology, detailing the metrics used to compare the retrieval configurations. Section 5.3 outlines the deterministic validation gate and the citation grounding metrics. Sections 5.4 and 5.5 present the empirical retrieval results and end-to-end performance metrics, respectively. Section 5.6 illustrates practical system behaviors using selected case studies, and Section 5.7 discusses error categories, system limitations, and avenues for future work.

5.2 Retrieval Evaluation Methodology

Evaluating the retrieval layer is a key step in assessing the system’s ability to locate appropriate legal evidence. In legal question answering, semantic search engines must do more than identify passages with general lexical or conceptual similarity; they must locate the exact statutory articles, clauses, and implementing provisions required to form a complete and legally sound response. The primary objective of the retrieval evaluation is to measure how effectively different retrieval architectures recover the precise legal coordinates necessary to address the benchmark queries.

To isolate the impact of the relational knowledge graph and the sparse indexing layers, we compare the retrieval effectiveness of three distinct configurations over the identical 69-query benchmark set, denoted as Q . The first configuration is *vector-only retrieval*, which relies entirely on dense semantic embeddings generated by the SBERT model, evaluating whether continuous vector representations alone are sufficient to map user requests to corresponding legal passages. The second configuration is *hybrid retrieval*, which fuses dense semantic vectors with sparse lexical tokens and discrete statutory coordinate metadata. This mode executes multi-query parallel prefetching and merges candidate lists using Reciprocal Rank Fusion (RRF) to examine whether combining semantic generalization with exact-match token structures improves retrieval coverage. The third configuration is the proposed *graph-augmented retrieval* framework, which extracts the high-relevance search hits from the hybrid layer to serve as seed nodes S_k , and subsequently traverses the Neo4j relational network. This mode navigates parent-child hierarchies, cross-document reference edges, rule-based taxonomy links, and normative validity hierarchies to evaluate whether topological expansion recovers implementing regulations and technical schedules where surface lexical or embedding overlap is insufficient.

In the legal domain, traditional text-similarity metrics (such as BLEU or ROUGE) and stochastic language model relevance judgments (LLM-as-Judge) are often inadequate because a retrieved passage can be semantically close yet legally inapplicable or obsolete. Consequently, our retrieval evaluation methodology relies on objective, deterministic citation-based metadata matching. A retrieved evidence chunk d is verified as relevant if and only if its structural citation metadata exactly matches the manually annotated gold legal citation labels.

Let $q \in Q$ represent a single query within the evaluation benchmark, and let $G(q)$ represent the set of required ground-truth legal citations necessary to fully ground the answer for q . The retrieved context sequence generated for query q under a top- k constraint is denoted as $R_k(q)$. Defining $C(d)$ as the unique canonical citation surface encapsulated within chunk d , the complete set of retrieved citation addresses is formalized as:

$$\mathcal{C}_k(q) = \{C(d) \mid d \in R_k(q)\}$$

Using this mathematical foundation, we define two primary recall-oriented retrieval metrics. First, $\text{Recall}@k(q)$ computes the proportion of ground-truth legal citations successfully recovered within the top- k context window for a single query:

$$\text{Recall}@k(q) = \frac{|G(q) \cap \mathcal{C}_k(q)|}{|G(q)|}$$

The macro-averaged retrieval recall across the entire diagnostic benchmark query set Q is formulated as the arithmetic mean:

$$\text{Recall}@k = \frac{1}{|Q|} \sum_{q \in Q} \text{Recall}@k(q)$$

In this study, empirical metrics are reported at baseline constraints of $k = 5$ and $k = 10$.

While fractional recall measures gradual improvement, legal compliance typically requires full evidence capture. Therefore, we implement a more stringent, binary information retrieval metric termed *Required Citation Coverage*, which checks whether the retriever successfully establishes the complete set of required gold citations without a single omission. A query passes this check if and only if the ground-truth set is fully contained within the retrieved space, represented mathematically via the indicator function $\mathbb{I}[\cdot]$:

$$\text{Coverage}(q) = \mathbb{I}[G(q) \subseteq \mathcal{C}_k(q)]$$

The macro-averaged Required Citation Coverage across the benchmark distribution is subsequently defined as:

$$\text{RequiredCitationCoverage} = \frac{1}{|Q|} \sum_{q \in Q} \text{Coverage}(q)$$

To prevent the retrieval of legally obsolete, repealed, or highly misleading contexts that jeopardize system safety, the benchmark incorporates explicit negative annotations. Let $F(q)$ define the set of annotated forbidden citations for query q . The *Forbidden Citation Violation Rate* measures how frequently a retrieval configuration mistakenly introduces these unauthorized legal references into the active prompt window, compromising generation safety. The violation indicator for a single query is formalized as:

$$\text{ForbiddenViolation}(q) = \mathbb{I}[F(q) \cap \mathcal{C}_k(q) \neq \emptyset]$$

The macro-averaged Forbidden Citation Violation Rate across the experimental distribution is formulated as:

$$\text{ForbiddenViolationRate} = \frac{1}{|Q|} \sum_{q \in Q} \text{ForbiddenViolation}(q)$$

To deliver a comprehensive information retrieval profile, our evaluation environment also documents Precision@ k and the Mean Reciprocal Rank (MRR), utilizing standard mathematical formulations over the citation match set. Precision@ k computes the proportion of retrieved chunks matching valid gold citations, while MRR evaluates the reciprocal of the ordinal rank of the first correct statutory chunk returned.

In production legal QA architectures, Recall@ k and Required Citation Coverage are heavily prioritized over Precision@ k . While a low precision score merely denotes the

presence of superfluous supporting context (which can be programmatically filtered by downstream generation and reranking components), a low recall score signifies an unrecoverable failure. Specifically, if mandatory statutory provisions are entirely missing from the context window, the downstream generation of a citation-grounded answer becomes extremely difficult. The empirical comparison of these retrieval configurations across different queries, difficulty levels, and functional categories is evaluated in Section 5.4.

5.3 Answer and Citation Evaluation Methodology

Evaluating the final output of a legal question-answering assistant requires strict, multi-tiered assessment criteria. In regulatory and statutory contexts, a generated response must do more than demonstrate linguistic fluency or logical coherence; it must be grounded within verified legislative sources and free of hallucinated citations or coordinate mentions. To enforce these foundational constraints, the proposed architecture incorporates an automated, closed-loop validation gate. The primary objective of this evaluation methodology is to analyze the qualitative precision of the generated answers and verify the structural validity of their underlying legal citation surfaces against our diagnostic benchmark.

Answer quality is evaluated using deterministic rule-based validation, not human legal expert review or LLM-as-Judge evaluation. In high-risk regulatory tasks, using stochastic language models as evaluators introduces unacceptable instability, prompt dependency, and potential evaluation-level hallucinations into the performance metrics themselves. Conversely, continuous human expert evaluation remains prohibitively expensive for iterative developer testing. By formulating explicit, programmatic verifications over citation payloads, article numerals, and text densities, we establish an objective, auditable, and reproducible verification pipeline. This deterministic validation engine isolates and measures five core operational dimensions: retrieval success, answer presence, citation grounding, substantive quality, and end-to-end pipeline pass rates.

Citation grounding. To ensure that every legal claim generated by the system is explicitly bound to verified statutory sources, the validation layer executes automated citation tracing. Let q define a user query, let C_q represent the consolidated set of retrieved context chunks passed to the generator, and let B_q define the set of canonical legal basis citations utilized within the final response payload. Defining $\mathcal{C}(C_q)$ as the definitive universe of all valid statutory citation surfaces available within the retrieved contexts, the system evaluates the citation grounding check via strict set-theoretic subset containment:

$$B_q \subseteq \mathcal{C}(C_q).$$

This containment check acts as an architectural boundary, blocking the system from referencing external or unverified provisions. If the response introduces a citation lying outside the actively retrieved context space, the check fails.

Beyond explicit citation indexing, the framework evaluates the structural authenticity of all numerical coordinates present within the natural language body text to suppress coordinate hallucinations. Let A_{answer} define the set of all unique article numbers explicitly extracted from the generated answer string using regular expressions, and let A_{contexts} represent the complete set of article numbers embedded within the retrieved context chunks C_q . The system enforces the article numeral support constraint as:

$$A_{\text{answer}} \subseteq A_{\text{contexts}}.$$

If the model synthesizes a phantom article number or incorrectly references a coordinate absent from its input context space, this check fails, capturing instances of internal structural drift. Furthermore, to satisfy the negative constraints established within the diagnostic benchmark, the engine validates the response against the query’s annotated forbidden citation set, denoted as $F(q)$. The forbidden citation check is formulated as:

$$F(q) \cap B_q = \emptyset.$$

This check helps ensure that the answer does not rely on legally obsolete, repealed, or distracting provisions, even if those provisions exhibit high semantic proximity to the initial query.

Answer quality checks. The validation gate implements direct text-density profiling to verify that the generated output is substantive, informative, and directly resolves the user’s intent. To prevent the model from outputting low-information responses—such as empty citation pointers or verbatim repetitions of article titles devoid of statutory content—the engine enforces an adversarial text-density check. If the natural language text length or the accompanying evidence quotes fall below a token threshold of 8, or if the string contains metadata patterns without explicit statutory text, the quality gate fails, indicating a low-information or incomplete response.

In simple language, answer quality checks ensure that:

- the answer must directly address the question,
- it must include the required legal rule,
- it must not only repeat article titles,
- it must avoid low-information quotes,
- numeric or yes/no questions must include the expected type of answer when applicable.

Authority and insufficient-context checks. To reflect the vertical hierarchy of Vietnamese legislation, the evaluation pipeline incorporates structural authority verification. Because a lower integer rank in our data schema maps to a higher legal validity tier (Code > Decree > Circular), the validator ensures the response does not skip active primary statutes when they are available within the retrieval window. Let $r_{\text{available}}^{\min}$ represent the minimum numeric rank present within C_q , and let $r_{\text{cited}}^{\min}$ define the minimum numeric rank extracted from the answer’s legal basis B_q . The rank compliance check checks that:

$$r_{\text{cited}}^{\min} \leq r_{\text{available}}^{\min}.$$

This checks that the answer does not cite low-rank circulars while ignoring primary code acts present in its context space. Finally, to address edge cases where the retriever fails to discover adequate evidence, the engine assesses the uncertainty disclosure check. If the context is flagged as insufficient, the system verifies that the response explicitly incorporates predefined uncertainty indicators to avoid the release of speculative legal assertions.

End-to-end pass logic. The comprehensive performance evaluation of a query $q \in Q$ within the closed-loop system is mathematically formalized as the strict logical conjunction of all independent validation dimensions. Let $\text{Pass}_{\text{retrieval}}(q)$ define the boolean recall status of the retrieval layer (requiring a non-zero recall capture), and let $\text{Pass}_{\text{answer}}(q)$, $\text{Pass}_{\text{citation}}(q)$, and $\text{Pass}_{\text{quality}}(q)$ represent the respective boolean states returned by the

answering, citation, and quality guardrails. Additionally, the required citation check verifies that the required citations are successfully retrieved:

$$G(q) \subseteq \mathcal{C}(C_q).$$

The definitive end-to-end system pass state, denoted as $\text{Pass}_{\text{E2E}}(q)$, is computed via:

$$\text{Pass}_{\text{E2E}}(q) = \text{Pass}_{\text{retrieval}}(q) \wedge \text{Pass}_{\text{answer}}(q) \wedge \text{Pass}_{\text{citation}}(q) \wedge \text{Pass}_{\text{quality}}(q).$$

This formulation dictates that a query is recorded as a complete success in the end-to-end pipeline if and only if every single retrieval, citation, structural, and quality gate passes simultaneously. A failure in a single guardrail invalidates the entire operational sequence.

This deterministic validation approach possesses significant advantages: it is completely independent of the stochastic volatility and bias associated with neural evaluators, it enforces transparency, and it supports micro-level auditability. However, its boundaries must be noted: it does not substitute for the qualified counsel of professional legal experts, it measures compliance strictly relative to the bounded indexed corpus, it treats correctness as an exact alignment with the constructed diagnostic benchmark, and it may miss semantic nuance. The quantitative results derived from this evaluation methodology across all difficulty levels and functional categories are presented and analyzed in Sections 5.4 and 5.5.

5.4 Retrieval Results

This section evaluates and compares the retrieval effectiveness of vector-only, hybrid, and graph-augmented retrieval modes using citation-based metrics on the constructed 69-query benchmark. The primary evaluation metrics, average Recall@5, average Recall@10, Precision@5, Precision@10, Mean Reciprocal Rank (MRR), Required Citation Coverage, and the Forbidden Citation Violation Rate, are reported across all configurations in Table 5.2. To analyze functional retrieval characteristics under varying legal requirements, Table 5.3 breaks down the Recall@10 and Required Citation Coverage performance by query category.

Table 5.2: Retrieval modes comparison table.

Mode	R@5	R@10	P@5	P@10	MRR	Coverage	Forbidden
Vector-only	0.012	0.012	0.026	0.020	0.022	0.012	0.000
Hybrid	0.860	0.920	0.571	0.380	0.833	0.920	0.000
Graph-augmented	0.961	1.000	0.562	0.359	0.872	1.000	0.000

As shown in Table 5.2, vector-only retrieval performed poorly on this benchmark, achieving a Recall@10 of only 0.012 and Required Citation Coverage of 0.012. This suggests that relying solely on continuous dense embeddings is insufficient for legal QA. Vietnamese legal evidence frequently depends on exact coordinates, such as specific article numbers, clause letters, and appendix indices. While vector similarity model embeddings excel at broad data grouping and identifying general legal topics, they often miss these precise coordinate identifiers. Furthermore, chunks that are semantically similar to a user’s question might be legally inapplicable, belong to repealed provisions, or reference

the wrong legislative act. Consequently, semantic proximity alone does not guarantee the recovery of correct legal bases.

In contrast, hybrid retrieval showed a major improvement, raising Recall@10 to 0.920 and Required Citation Coverage to 0.920. Fusing dense semantic vectors with sparse tokens using Reciprocal Rank Fusion (RRF) combines the advantages of both retrieval models. The dense encoder handles synonym expansion and paraphrased user queries, while the keyword search ensures that specific legal terms, document identifiers, and statutory coordinates are matched exactly. This hybrid configuration successfully matches direct queries but experiences limitations on queries requiring cross-document references or complex tabular lookups, as shown in Table 5.3.

The graph-augmented retrieval configuration achieved the highest performance on this benchmark, reaching a Recall@10 of 1.000 and Required Citation Coverage of 1.000. Starting from hybrid seed chunks, the graph retriever traverses structural, reference, taxonomy, and normative validity edges to expand the context window. This topological expansion helps recover subordinate circulars, procedural conditions, and related provisions that may not appear in the top-ranked hybrid results. As shown in the category-level results in Table 5.3, this approach especially improves performance in complex categories such as document guidance (Recall@10 improved from 0.683 to 1.000), multi-hop questions (Recall@10 improved from 0.583 to 1.000), and procedure questions (Recall@10 improved from 0.857 to 1.000). The graph traversal ensures that multi-tier relationships (linking a Code to administrative Decrees and ministerial Circulars) are fully navigated, which is crucial for solving calculation, table lookup, and exceptions queries.

Table 5.3: Category-level result table.

Category	Q	Vec R@10	Hyb R@10	Graph R@10	Hyb Cov.	Graph Cov.
Calculation/table lookup	5	0.067	1.000	1.000	1.000	1.000
Comparison QA	4	0.000	1.000	1.000	1.000	1.000
Definition QA	7	0.000	1.000	1.000	1.000	1.000
Direct QA	19	0.000	1.000	1.000	1.000	1.000
Document guidance QA	10	0.000	0.683	1.000	0.683	1.000
Exception-based QA	11	0.045	0.955	1.000	0.955	1.000
Multi-hop QA	2	0.000	0.583	1.000	0.583	1.000
Procedure QA	7	0.000	0.857	1.000	0.857	1.000
Scenario-based QA	4	0.000	1.000	1.000	1.000	1.000

Importantly, the forbidden citation violation rate remained at 0.000 across all modes, indicating that graph expansion did not introduce benchmark-defined forbidden citations in this experiment. This suggests that the graph schemas and traversal filters effectively control prompt noise. However, this does not suggest that the retriever is legally perfect under all scenarios, nor does it guarantee that irrelevant legal information will never be retrieved. These results are limited to the constructed 69-query benchmark and six-document labor corpus, and do not prove universal correctness for all potential labor law queries. Precision@10 decreased slightly in the graph mode (from 0.380 to 0.359) because expanding context via the graph introduces additional supporting nodes; however, this minor trade-off is acceptable since high recall is required for downstream generation. Indeed, on the subset of 25 graph-required queries, the graph expansion enabled a 1.000 retrieval pass rate, which provided a complete evidence base for subsequent steps.

With the retrieval stage establishing a fully grounded context window under the graph-augmented configuration, the next section evaluates the end-to-end performance and the validation checks of the answer generation layer.

5.5 End-to-End Results

This section presents the results of the complete end-to-end pipeline evaluation, which traces the system’s execution from initial retrieval, through context assembly and answer generation, to the final deterministic validation gates. The primary objective is to verify whether the integrated pipeline can produce high-quality, citation-grounded responses that pass all structural and content guardrails. The quantitative results of this evaluation over the 69-query diagnostic benchmark are summarized in Table 5.4.

Table 5.4: End-to-end evaluation summary table.

Metric	Value
Benchmark queries	69
End-to-end pass rate	1.000
Retrieval pass rate	1.000
Answer pass rate	1.000
Citation validation pass rate	1.000
Quality validation pass rate	1.000
Low-information quotes	0
Unsupported article numbers	None
Unretrieved citations	None

As shown in Table 5.4, the system achieved a 1.000 pass rate across the end-to-end pipeline, the retrieval layer, answer presence, citation validation, and quality validation on the 69 benchmark queries. Under the deterministic validation rules, all generated answers successfully satisfied the required structural constraints. It is critical to interpret these results within the correct scope: a 1.000 pass rate indicates that the system met all rule-based constraints for this constructed benchmark within the indexed six-document corpus. These results do not suggest that the system is perfectly correct in all scenarios, nor do they guarantee absolute legal correctness or represent legally certified answers.

Regarding citation grounding, the evaluation did not detect any unsupported article numbers or unretrieved citations in the generated responses. The article numeral verification check confirmed that all article numbers referenced in the natural language text were subset within the retrieved contexts. This indicates that the legal bases generated by the answering layer matched the retrieved evidence and did not introduce external citations or fabricated coordinates. Ensuring exact coordination is essential in legal QA because a technically correct statement referencing a wrong or unretrieved article introduces significant compliance risks.

Furthermore, the quality checks did not detect any low-information quotes in the final evaluations. The extractive generation and validation filters prevented the model from returning empty text fields, verbatim title repetitions, or excessively short evidence snippets. The average quality score reached 100.00 under the rule-based metrics, suggesting that the generated answers provided substantive retrieved legal provisions to address the queries rather than relying solely on citation labels. These results indicate that the pipeline worked consistently across all difficulty groups (comprising 15 easy, 32 medium, and 22 hard queries), category-level functional splits, and normative-hierarchy-required queries (22 true and 47 false queries) on the constructed benchmark.

However, these results are limited to the constructed 69-query benchmark and the indexed six-document corpus. It is important to emphasize that this deterministic validation is a software-level verification step and does not replace professional legal expert review. While rule-based constraints are highly reproducible and useful for identifying structural errors, they do not evaluate semantic nuances or guarantee that the output remains legally correct under changing regulatory frameworks.

With the end-to-end performance established on the benchmark, the next section presents specific case studies to illustrate practical system behaviors and context-grounding flows.

5.6 Case Studies

This section presents selected benchmark cases to illustrate how the retrieval and validation pipeline behaves. The examples are not additional evaluation results; they are qualitative explanations of representative benchmark items.

Case 1: Minor worker employment. The benchmark query asks: “Hãy viết giải thích về lao động chưa thành niên cho một chủ hộ kinh doanh đang muốn thuê người 14 tuổi làm việc.” This query belongs to the `minor_labor` category and requires Bộ luật Lao động 2019, Điều 143, Điều 144, Điều 145. A vector-only or hybrid retriever may miss the specific implementing circular guidelines (such as Circular 09/2020/TT-BLĐTBXH) that list the exact allowed light work occupations. Because the query’s semantic encoding centers on the general concept of minor labor, similarity-based search matches broad Labor Code sections but fails to traverse down the administrative hierarchy. The graph-augmented retriever helps by traversing GUIDED_BY and DETAILS reference edges from the parent Labor Code Article 143 seed node to the implementing circular chunks. This topological traversal fetches the precise allowed light work occupations for children aged 13 to 15. The final answer is accepted because the deterministic citation check confirms all utilized legal bases are grounded within the retrieved Circular context. The pipeline successfully outputs the statutory conditions, such as obtaining parental consent and ensuring that the work does not interfere with the minor’s schooling.

Case 2: Retirement age table lookup. The benchmark query asks: “Nu nghỉ hưu năm 2026 thì bao nhiêu tuổi và căn cứ văn bản nào?” This query belongs to the `calculation_or_table_lookup` category and requires Bộ luật Lao động 2019, Điều 169 và Nghị định 135/2020/NĐ-CP, Điều 4 và PHỤ LỤC I. A vector-only or hybrid retriever may miss the exact retirement age table row because dense embeddings struggle with numeric tables. While hybrid search retrieves the general Article 169 clause, it often fails to rank the specific female retirement table chunk high enough to enter the prompt context. The graph-augmented retriever helps by traversing the DETAILS reference edge from Article 169 of the Labor Code directly to Article 4 and Appendix I of Decree 135/2020/NĐ-CP. This pulls the exact female retirement table mapping year 2026 to 57 years of age. The final answer is accepted because the citation check verifies that the output correctly cites the specific column and row in Appendix I, validating that the generated answer is grounded in the retrieved table. The response accurately states that the female retirement age in 2026 is 57 years.

Case 3: Labor contract content guidance. The benchmark query asks: “Hợp đồng lao động cần nội dung gì và thông tin nào hướng dẫn chi tiết?” This query belongs to the `document_guidance_qa` category and requires Bộ luật Lao động 2019, Điều 21

và Thông tư 10/2020/TT-BLĐTBXH, Điều 3. A vector-only or hybrid retriever may miss the specific implementing ministerial guidelines in the Circular because dense vector proximity models rank parent Code articles higher due to high keyword overlap. This leaves the downstream answering layer without the granular definitions for salary, allowance, and payment terms. The graph-augmented retriever helps by traversing the GUIDED_BY edge from Labor Code Article 21 to Article 3 of Circular 10/2020/TT-BLĐTBXH. This automatically expands the context to include the detailed administrative instructions. The final answer is accepted because the deterministic validation gates verify that both the parent Code and the guiding Circular are retrieved and cited together. The output lists all required contract elements while ensuring that the detailed guidelines from the Circular are properly incorporated and cited.

Case 4: Dismissal dispute or labor procedure question. The benchmark query asks: “Tranh chấp sa thải có cần hòa giải trước khi kiện không và Tòa án có thẩm quyền không?” This query belongs to the multi_hop_qa category and requires Bộ luật Lao động 2019, Điều 188 và Điều 190, và Bộ luật Tố tụng dân sự 2015, Điều 32. A vector-only or hybrid retriever may miss the procedural links between separate legislative codes. Keyword similarity search struggles to connect BLLĐ Article 188 (which exempts dismissal disputes from mandatory mediation) to BLTTDS Article 32 (which establishes court jurisdiction over labor disputes) due to their disjointed phrasing. The graph-augmented retriever helps by traversing cross-document REFERENCES edges between BLLĐ Article 188 and BLTTDS Article 32. This gathers both the substantive labor exemption and the civil procedural jurisdiction rules into the same prompt window. The final answer is accepted because the citation validation step checks that BLLĐ Article 188 and BLTTDS Article 32 are present in the response. The extractive answering layer successfully delivers a grounded response explaining that dismissal disputes are exempt from mediation and fall under direct court jurisdiction.

These cases suggest why graph expansion is useful for cross-document and hierarchy-dependent questions. However, they should be interpreted together with the quantitative results in Sections 5.4 and 5.5, not as proof of universal legal correctness.

5.7 Error Analysis and Limitations

Benchmark scope. The quantitative results presented in Sections 5.4 and 5.5—including the 1.000 Recall@10, the 1.000 Required Citation Coverage, and the 1.000 end-to-end pass rate—demonstrate the empirical performance of the graph-augmented RAG system on the constructed diagnostic benchmark. However, it is critical to recognize that these findings are bound by the scope of this evaluation. The benchmark contains 69 constructed queries, which, while highly detailed and useful for controlled evaluation, do not constitute a full, real-world legal question-answering benchmark. This diagnostic set is designed to test specific failure modes (such as document routing, exceptions, and cross-document links) but does not cover every possible Vietnamese labor-law question or user scenario. The benchmark is sufficient for evaluating the implemented system behavior, but it should not be interpreted as a complete test of Vietnamese labor-law reasoning. Because these queries were heuristically designed for system diagnostics rather than collected from live, diverse user interactions, they likely underrepresent the linguistic variations, conversational noise, and ambiguous intents that occur in practical deployments.

Corpus coverage. A major structural constraint is that the system’s knowledge

is bounded by the six indexed legal documents. The retriever and generator can only perform successfully when the required legal evidence exists within this scoped corpus. Questions involving laws or regulations outside the indexed documents—such as tax codes, business enterprise laws, or newer ministerial circulars—cannot be resolved and will require external sources. Furthermore, the cross-reference graph construction process made conservative choices during parsing: 111 unresolved references were identified and deliberately excluded from the active Neo4j graph. This is a conservative choice to avoid adding uncertain graph edges, but it also means the graph does not capture every legal reference. While this prevents the retrieval pipeline from traversing invalid or dead-end paths, it restricts the query expansion policy from navigating long-tail reference networks that might be necessary for broader regulatory integration.

Evaluation method. The approach used to validate the generated answers also introduces specific limits. Answer quality is evaluated using deterministic rule-based validation, not human legal expert review or LLM-as-Judge evaluation. The deterministic validation gates—checking for direct answer presence, exact citation matching, and the absence of low-information or verbatim repeats—ensure high reproducibility, easy inspection, and transparent auditing of the system’s responses. However, these software-level checks do not replace professional legal expert review. Rule-based metrics are highly effective at catching structural anomalies, incorrect article numbers, or missing citations, but they are fundamentally blind to semantic nuances, logical flow, and subtle errors in legal interpretation. Passing all validation checks indicates that the output satisfies the structural rules defined in the benchmark, not that the legal reasoning would be accepted by a practicing lawyer or a court of law.

Generation and retrieval limits. The generation and retrieval layers also exhibit specific operational boundaries. The answer generator relies on a highly structured, extractive, and citation-grounded template to assemble response components. This design is highly effective at reducing unsupported citation claims and preventing hallucinated article numbers. However, this extractive style can make the final output less flexible when explaining complex legal arguments, synthesising contrasting opinions, or answering open-ended, multi-clause scenario questions. Additionally, the graph-augmented retrieval process is highly sensitive to the quality of the upstream data engineering. The performance of graph expansion depends entirely on chunking quality, reference extraction, metadata enrichment, and graph schema design. If a citation is unresolved, outside the graph, or parsed incorrectly, the query expansion policy cannot recover the missing context, which directly limits the downstream generator.

Practical use. In light of these constraints, the system should be treated as an intelligent legal information assistant within the indexed corpus rather than a substitute for professional legal advice. In real-world deployments, answers must include clear user-facing disclaimers, and high-stakes use cases should incorporate a human-in-the-loop expert review process. To move beyond the limits of this study, future research should focus on expanding the indexed corpus to a broader set of civil and administrative statutes and developing an expert-validated benchmark. Integrating real user query logs and establishing a rigorous human legal review panel will help evaluate conversational adaptability and semantic accuracy. Additionally, comparing the rule-based validation framework with advanced LLM-as-Judge evaluations and introducing mechanism updates to handle temporal validity when legal documents change will be valuable paths forward.

These limitations do not invalidate the experimental results. Instead, they define the scope in which the results should be interpreted: a controlled, citation-based evaluation

of a graph-augmented RAG system over a bounded Vietnamese labor-law corpus.

Chapter 6

Discussion

6.1 Key Findings

The empirical results compiled across the experimental evaluations demonstrate the effectiveness of combining structured data processing, hybrid indexing, and relational graph expansion. On the constructed 69-query benchmark and within the indexed six-document corpus, the proposed system achieved complete retrieval coverage and a 1.000 end-to-end pass rate under strict, deterministic validation gates. The primary findings indicate that hierarchy-aware chunking produces highly clean and structurally intact evidence units that preserve legal meaning. Furthermore, graph-augmented retrieval successfully addresses the limits of semantic and keyword search by traversing administrative ties, significantly improving required citation coverage. Finally, the automated validation layer provides a reliable, reproducible, and transparent mechanism to check answer grounding and citation validity, verifying that every legal assertion is backed by retrieved source evidence.

6.2 Impact of Hierarchical Chunking

Granular evidence localization is a major factor in legal information retrieval, where arbitrary character-limit or token-limit sliding windows frequently fragment statutory provisions. Hierarchical chunking by Article, Clause, and Point preserves the structural unity of legal texts, ensuring that provisions are never bisected or separated from their primary contextual anchors. By maintaining parent-child relations (such as linking a specific Point to its parent Clause and parent Article), the data processing pipeline ensures that the retriever retains crucial environmental details and legislative context. Consequently, when a specific clause is retrieved, the system can preserve and format the full citation path, enabling precise downstream citation validation and preventing the loss of adjacent qualifying terms or exceptions.

6.3 Benefits of Graph-Augmented Retrieval

The comparative analysis of retrieval modes in Chapter 5 highlights the advantages of traversing relational legal networks. While hybrid retrieval achieves a strong Recall@10 of 0.920 on the 69-query benchmark, the graph-augmented retriever reaches a Recall@10 of

1.000 and 1.000 required citation coverage. Rather than relying solely on semantic proximity or keyword matching, which struggle when legal concepts are expressed through disparate lexical terms, the graph retriever utilizes hybrid search hits as seed nodes and traverses structural, taxonomy, and reference edges. This relational expansion is especially beneficial for document guidance queries, multi-hop questions, administrative table lookups, and questions requiring the coordination of parent Code acts with implementing Decrees and ministerial Circulars. However, these improvements are benchmark-specific and do not suggest that graph traversal always improves every possible legal query, particularly where user questions map directly to isolated, self-contained statutory articles.

6.4 Role of Normative Hierarchy in Legal QA

Vietnamese legislation is organized in a strict, vertical normative hierarchy where primary laws (such as the Labor Code) establish broad regulatory regimes, while executive Decrees and ministerial Circulars define administrative details, exceptions, and procedural guidelines (i.e., Code/Law > Decree > Circular). The graph model incorporates these normative ranks as discrete properties and directed **GUIDED_BY** edges, enabling the system to order retrieved context chunks based on their legislative authority. By enforcing authority constraints, the retriever prioritizes primary statutes over lower-level administrative guidance when both are available in the retrieved window. This hierarchical structuring prevents the generator from relying exclusively on lower-level circulars while neglecting parent code provisions, thereby aligning the system’s context assembly with standard legal interpretation principles.

6.5 Citation Grounding and Hallucination Control

Hallucination represents a major safety concern when deploying large language models in regulatory and statutory environments, where fabricated citations or phantom article numbers can lead to serious compliance violations. The implemented architecture addresses this challenge by running the generated response payload through a closed-loop, rule-based validation gate. This gate performs set-theoretic checks to verify that every citation and numerical coordinate present in the answer is present within the actively retrieved context blocks. By restricting the generator’s legal basis references to verified source chunks and deploying an extractive fallback when LLM validation checks fail, the system significantly reduces unsupported legal references and coordinate drift. However, while these deterministic validation rules effectively detect and block ungrounded assertions on the benchmark queries, they do not completely eliminate the risk of hallucination under all potential generation conditions.

6.6 Practical Applications

The modular, validated nature of the proposed system makes it well-suited for several practical information retrieval use cases. It can serve as a legal information assistant to support academic study, student research, and quick statutory retrieval for legal researchers. Additionally, the system provides a valuable lookup utility for internal corporate human resource (HR) compliance, enabling HR personnel to quickly identify relevant

sections of the Labor Code, retirement schedules, or contract guidelines. However, despite its strong performance on the diagnostic benchmark, the system is designed to be an information lookup aid and must not be treated as a substitute for professional legal advice or qualified legal counsel. Real-world deployments should always feature explicit user-facing disclaimers and incorporate a human-in-the-loop verification step for high-stakes regulatory decisions.

6.7 Limitations

The system achieved 1.000 pass rates on the constructed benchmark, but this should be interpreted only within that benchmark and corpus. While these results reflect strong structural and citation safety, they do not prove that the system is legally perfect across all possible labor-law queries. The primary limitation of this study lies in the scope of the corpus, which contains only six indexed Vietnamese labor-law documents, and the evaluation benchmark, which consists of 69 constructed diagnostic queries. These queries were designed to test specific system capabilities rather than capture the full, messy spectrum of live user interactions, linguistic errors, and ambiguous intent. Furthermore, the system has not been subjected to full, double-blind legal expert validation, and the rule-based validation gate—while highly objective and reproducible—may miss subtle semantic errors, logical inconsistencies, or nuances in legal interpretation.

Extractive answers represent another operational trade-off, as they prioritize strict citation safety at the cost of conversational fluency and natural language adaptability. When the extractive fallback is triggered, the system returns exact statutory passages, which are highly accurate but can feel rigid compared to conversational human responses. Additionally, the system’s performance remains highly dependent on upstream data engineering quality; any errors in hierarchical chunking, metadata enrichment, or cross-reference extraction will propagate down to the retrieval and answering layers. Finally, the conservative design choice to exclude 111 unresolved references outside the corpus restricts the graph query expansion policy from navigating broader legislative networks, a limitation that must be addressed before the system can scale to cover wider civil and administrative legal domains.

Table 6.1: Limitations summary.

Limitation	Thesis Treatment
Scoped corpus	Covers six indexed legal documents and does not cover all Vietnamese law.
Constructed benchmark	Interpret 1.000 scores as benchmark-specific evidence only.
Deterministic validation	Quality was not reviewed by human legal experts or LLM-as-Judge.
Extractive answers	Extractive answers improve grounding but may be less fluent or adaptive.
Upstream parsing dependence	Chunking, metadata, and reference extraction errors can affect retrieval.
Legal updates	Newer or external legal sources require corpus refresh and re-evaluation.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

TODO: Conclude with evidence-backed claims only: the implemented system integrates hierarchy-aware chunking, Qdrant, Neo4j, graph-augmented retrieval, extractive answer generation, and deterministic citation validation for a scoped Vietnamese labor-law corpus.

7.2 Future Work

TODO: Introduce concrete next steps that address the current benchmark and validation limitations.

- **Expert-validated benchmark.**

TODO: Create a benchmark reviewed by Vietnamese labor-law experts and compare it with the current constructed benchmark.

- **LLM-as-Judge evaluation.**

TODO: Add LLM-as-Judge evaluation only as a future complementary method with strict grounding checks, not as the current final evaluation method.

- **Human legal review.**

TODO: Evaluate answer usefulness, legal nuance, and risk with qualified legal reviewers.

- **Larger legal corpus.**

TODO: Expand the corpus to additional labor-related laws, implementing documents, official guidance, and newer amendments.

- **Improved generative answering.**

TODO: Explore constrained abstractive generation while preserving citation grounding, refusal behavior, and article-number validation.

- **Real-world deployment.**

TODO: Study production requirements such as legal update monitoring, user feedback, audit logs, privacy, latency, and responsible-use warnings.

References

- [1] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. 2020.
- [2] TODO. *TODO: Verified official sources for Vietnamese labor-law legal documents*. Placeholder citation key. Replace with official legal-document citation metadata before final submission.
- [3] TODO. *TODO: Verified reference for GraphRAG or graph-augmented retrieval*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [4] TODO. *TODO: Verified reference for legal question answering*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [5] TODO. *TODO: Verified reference for Qdrant vector database*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.
- [6] TODO. *TODO: Verified reference for Sentence Transformers*. Placeholder citation key. Replace with verified bibliographic metadata before final submission.

Appendix A

List of Legal Documents

This appendix provides a complete list of the six indexed Vietnamese legal documents comprising the scoped labor-law corpus. The corpus is structured across multiple legislative ranks, spanning primary codes, implementing decrees, and ministerial circulars.

Table A.1: Scoped Vietnamese labor-law corpus documents.

Document ID	Official Title	Document Type	Chunks
45-2019-qh14	Bộ luật Lao động 2019	bo_luat	697
92-2015-qh13-labor-only	Bộ luật Tổ tụng dân sự 2015 (subset)	bo_luat	159
ngghi-dinh-135-2020-nd-cp	Nghị định 135/2020/NĐ-CP	ngghi_dinh	41
ngghi-dinh-145-2020-nd-cp	Nghị định 145/2020/NĐ-CP	ngghi_dinh	520
thong-tu-09-2020-tt-blđtbbxh	Thông tư 09/2020/TT-BLĐTBXH	thong_tu	73
thong-tu-10-2020-tt-blđtbbxh	Thông tư 10/2020/TT-BLĐTBXH	thong_tu	66
Total Corpus			1,556

Appendix B

Chunk Schema

This appendix documents the object-oriented JSONL schema for preprocessed and enriched evidence units generated by the data processing pipeline. Each chunk encapsulates both hierarchical structural anchors and semantic enrichment attributes.

```
{
  "document_id": "45-2019-qh14",
  "document_title": "Bộ luật số 45/2019/QH14",
  "source_kind": "curated_text",
  "source_path": "corpus/data/curated/45_2019_QH14.txt",
  "chunk_id": "45-2019-qh14-dieu-2-chunk-01",
  "section_id": "45-2019-qh14-dieu-2",
  "article_number": "2",
  "article_title": "Đối tượng áp dụng",
  "heading": "Điều 2. Đối tượng áp dụng",
  "chapter_heading": "Chương I. NHỮNG QUY ĐỊNH CHUNG",
  "section_heading": null,
  "chunk_index": 1,
  "char_count": 121,
  "text": "Điều 2. Đối tượng áp dụng\n\n1. Người lao động, người học nghề, người\n↪ tập nghề...",
  "clause_ref": "1",
  "point_ref": null,
  "point_refs": [],
  "chunk_type": "clause",
  "parent_chunk_id": null,
  "level": "clause",
  "citation_text": "Bộ luật số 45/2019/QH14, Điều 2 (Đối tượng áp dụng), khoản 1",
  "retrieval_text": "Trong Chương I. NHỮNG QUY ĐỊNH CHUNG, Bộ luật số\n↪ 45/2019/QH14, Điều 2...",
  "page_content": "Trong Chương I. NHỮNG QUY ĐỊNH CHUNG, Bộ luật số 45/2019/QH14,\n↪ Điều 2...",
  "topic": [],
  "actor": ["nguoi_lao_dong"],
  "issue_type": []
}
```


Appendix C

Graph Schema

This appendix documents the labels, relationship types, properties, and constraints implemented within the Neo4j relational knowledge graph.

C.1 Ontology Labels and Relationships

- **Node Labels:** `Legal_Document`, `Legal_Article`, `Legal_Clause`, `Legal_Point`, `Legal_Appendix`, `Evidence_Chunk`, `Legal_Topic`, `Legal_Actor`, and `Legal_IssueType`.
- **Relationship Types:**
 - *Structural Hierarchy:* `HAS_ARTICLE`, `HAS_CLAUSE`, `HAS_POINT`, `HAS_APPENDIX`.
 - *Source Link:* `HAS_SOURCE_CHUNK` (connecting leaf-level legal anchors to `Evidence_Chunk` nodes).
 - *Cross-Reference:* `REFERENCES` (citations), `DETAILS` (qualifications), and `GUIDED_BY` (administrative delegations).
 - *Taxonomic Metadata:* `MENTIONS_TOPIC`, `APPLIES_TO_ACTOR`, `HAS_ISSUE_TYPE`.
 - *Normative Hierarchy:* `SUPERIOR_TO`, `MUST_COMPLY_WITH`.

C.2 Constraints and Indexes

To ensure structural integrity and quick lookups, unique node constraints are applied to properties:

```
CREATE CONSTRAINT legal_node_node_id IF NOT EXISTS
FOR (n:LegalNode) REQUIRE n.node_id IS UNIQUE;
```

```
CREATE CONSTRAINT evidence_chunk_chunk_id_unique IF NOT EXISTS
FOR (n:Evidence_Chunk) REQUIRE n.chunk_id IS UNIQUE;
```

```
CREATE INDEX legal_node_node_type IF NOT EXISTS
FOR (n:LegalNode) ON (n.node_type);
```

```
CREATE INDEX legal_document_document_id IF NOT EXISTS
FOR (n:Legal_Document) ON (n.document_id);
```

C.3 Diagnostic Cypher Queries

The following Cypher queries are executed by `scripts/inspect_legal_graph.py` to audit database counts:

```
// Count nodes by type
MATCH (n:LegalNode)
RETURN n.node_type AS node_type, count(*) AS count
ORDER BY node_type;
```

```
// Count relationships by type
MATCH ()-[r]->()
RETURN type(r) AS edge_type, count(*) AS count
ORDER BY edge_type;
```

Appendix D

Benchmark Samples

This appendix presents representative sample cases extracted from the constructed diagnostic evaluation benchmark.

D.1 Case LBR__001: Definition of Employee

- **Query:** Người lao động được định nghĩa như thế nào theo Bộ luật Lao động 2019?
- **Category:** Definition QA
- **Difficulty:** Easy
- **Required Citations:** *Bộ luật Lao động 2019, khoản 1 Điều 3*
- **Gold Answer:** Người lao động là người làm việc cho người sử dụng lao động theo thỏa thuận, được trả lương và chịu sự quản lý, điều hành, giám sát của người sử dụng lao động. Độ tuổi lao động tối thiểu của người lao động là đủ 15 tuổi...

D.2 Case LBR__010: Oral Contract Exception

- **Query:** Trường hợp nào có thể giao kết hợp đồng lao động bằng lời nói?
- **Category:** Exception-based QA
- **Difficulty:** Easy
- **Required Citations:** *Bộ luật Lao động 2019, khoản 2 Điều 14; khoản 2 Điều 18; điểm a khoản 1 Điều 145; khoản 1 Điều 162*
- **Gold Answer:** Hai bên có thể giao kết bằng lời nói đối với hợp đồng có thời hạn dưới 01 tháng. Tuy nhiên, ngoại lệ này không áp dụng đối với người dưới 15 tuổi, người giúp việc gia đình, hoặc nhóm lao động thông qua người ủy quyền.

Appendix E

Evaluation Metrics

This appendix provides the formal mathematical definitions of the evaluation metrics used to measure retrieval effectiveness and response compliance in Chapter 5.

E.1 Information Retrieval Metrics

Let Q represent the set of benchmark queries, let $G(q)$ represent the set of ground-truth legal citations for a query $q \in Q$, and let $\mathcal{C}_k(q)$ represent the set of citations retrieved within the top- k candidates.

Recall@k:

$$\text{Recall @}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|G(q) \cap \mathcal{C}_k(q)|}{|G(q)|}$$

Required Citation Coverage:

$$\text{RequiredCitationCoverage} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}[G(q) \subseteq \mathcal{C}_k(q)]$$

where $\mathbb{I}[\cdot]$ is the boolean indicator function.

Forbidden Citation Violation Rate: Let $F(q)$ define the set of legally obsolete or distracting citations for query q :

$$\text{ForbiddenViolationRate} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}[F(q) \cap \mathcal{C}_k(q) \neq \emptyset]$$

E.2 End-to-End System Pass Logic

The complete end-to-end pass state for query $q \in Q$, denoted as $\text{Pass}_{\text{E2E}}(q)$, is computed via:

$$\text{Pass}_{\text{E2E}}(q) = \text{Pass}_{\text{retrieval}}(q) \wedge \text{Pass}_{\text{answer}}(q) \wedge \text{Pass}_{\text{citation}}(q) \wedge \text{Pass}_{\text{quality}}(q)$$

where each term denotes the boolean outcome of the retrieval layer, the answer presence gate, the citation containment check, and the token density quality check.

Appendix F

Reproducibility Commands

This appendix documents the shell commands required to execute evaluations, rebuild indices, or regenerate database structures in PowerShell.

F.1 Rerun Retrieval-Mode Evaluation

```
.venv\Scripts\python.exe -X utf8 scripts/evaluate_retrieval_modes.py `
  --benchmark-path artifacts/evaluation/end_to_end_expanded_benchmark.jsonl `
  --output-dir artifacts/evaluation `
  --output-prefix retrieval_modes_expanded `
  --top-k 10 `
  --prefetch-limit 24 `
  --device cpu `
  --embedding-provider sentence_transformers
```

F.2 Rerun End-to-End Evaluation

```
.venv\Scripts\python.exe -X utf8 scripts/evaluate_end_to_end_rag.py `
  --benchmark-path artifacts/evaluation/end_to_end_expanded_benchmark.jsonl `
  --output-dir artifacts/evaluation `
  --output-prefix end_to_end_expanded `
  --comparison-summary-path
  ↪ artifacts/evaluation/retrieval_modes_expanded_summary.json `
  --top-k 10 `
  --prefetch-limit 24 `
  --max-answer-contexts 8 `
  --provider extractive `
  --device cpu `
  --embedding-provider sentence_transformers
```

F.3 Optional: Regenerate Qdrant Vector Index

```
.venv\Scripts\python.exe scripts/build_index.py `
  --chunk-file artifacts/chunks/legal_chunks_enriched.jsonl `
  --artifacts-dir artifacts `
  --dense-model keepitreal/vietnamese-sbert `
  --collection-name vietnamese_labor_law_chunks `
  --device cpu
```

F.4 Optional: Rebuild Neo4j Legal Graph

```
.venv\Scripts\python.exe scripts/build_legal_graph.py `
--chunks-path artifacts/chunks/legal_chunks_enriched.jsonl `
--reference-edges-path artifacts/graph/reference_edges.jsonl `
--summary-path artifacts/graph/legal_graph_build_summary.json `
--with-concepts `
--with-references `
--with-normative-hierarchy `
--reset
```

Appendix G

Example Retrieved Contexts and Answers

This appendix provides a concrete runtime trace illustrating a retrieved context block and the corresponding structured output payload generated by the extractive fallback.

G.1 Example Input Query

“Người lao động được định nghĩa như thế nào theo Bộ luật Lao động 2019?”

G.2 Retrieved Evidence Chunk

```
{
  "chunk_id": "45-2019-ql14-dieu-3-chunk-02",
  "citation_text": "Bộ luật số 45/2019/QH14, Điều 3 (Giải thích từ ngữ), khoản 1",
  "text": "1. Người lao động là người làm việc cho người sử dụng lao động \n theo
    ↳ thỏa thuận, được trả lương và chịu sự quản lý, điều hành, giám sát \n của
    ↳ người sử dụng lao động. Độ tuổi lao động tối thiểu của người \n lao động là
    ↳ đủ 15 tuổi, trừ trường hợp quy định tại Mục 1 Chương XI..."
}
```

G.3 Validated Generated Output

```
{
  "answer": "Căn cứ Bộ luật số 45/2019/QH14, Điều 3, khoản 1, người lao động \n
    ↳ được định nghĩa là người làm việc cho người sử dụng lao động theo thỏa
    ↳ thuận, \n được trả lương và chịu sự quản lý, điều hành, giám sát của người
    ↳ sử \n dụng lao động. Độ tuổi lao động tối thiểu là đủ 15 tuổi.",
  "legal_basis": ["Bộ luật số 45/2019/QH14, Điều 3 (Giải thích từ ngữ), khoản 1"],
  "evidence_quotes": [
    {
      "citation": "Bộ luật số 45/2019/QH14, Điều 3, khoản 1",
      "quote": "Người lao động là người làm việc cho người sử dụng lao động..."
    }
  ],
  "insufficient_context": false
}
```